

一 本 写 给 初 学 者 的 小 书

数字芯片设计入门

从开关到芯片的九堂课



从 0 与 1 出发，理解一块芯片如何思考

2026 年 · 第 1 版

前言

这本小书写给想要真正理解数字芯片的人：可能是刚接触硬件的学生，可能是想转向芯片方向的软件工程师，也可能只是一个好奇心旺盛的读者。它不打算面面俱到，而是选取数字芯片设计中最核心、最能体现本质的那一条主线，把它讲透。

这条主线是：一块芯片，如何从最简单的电子开关出发，一层一层地构造出能够计算、能够记忆、能够按剧本行事的复杂系统。读完本书，你将理解处理器、存储控制器这类数字芯片赖以工作的全部基本原理；剩下的，只是这些原理在更大规模上的重复与组合。

本书的写法

写作时我遵循了几条原则，它们也解释了本书为什么长成这个样子。

先问为什么，再讲是什么。每引入一个概念，先让你看到没有它时会遇到什么麻烦。例如，先体会“组合电路没有记忆”这个困境，再引出锁存器；先看到“透明的锁存器会让数据一冲到底”，再引出边沿触发。概念是对问题的回答，不是从天而降的定义。

从具体到抽象。每个抽象概念都配有具体的例子和图。图不是装饰，而是论证的一部分，请务必对照正文阅读。

补齐数学台阶。凡是用到读者不一定熟悉的数学概念，比如模运算、组合爆炸、指数衰减，书中都用“数学加油站”栏目就地补充，不需要你另查资料。

刻意重复。重要的思想会在不同章节以不同面貌反复出现。例如“抽象与契约”这个主题，会在数字抽象、触发器时序、RTL 设计中出现三次。这不是啰嗦，而是符合记忆规律的安排。

让你动脑。每章末尾有思考题，参考答案在附录 A。强烈建议先自己想，再看答案；提取式的回忆远比重读有效。

各章的路线图

全书自底向上，共九章。第 1 章回答最根本的问题：为什么用 0 和 1。第 2 章从晶体管开关搭出逻辑门。第 3 章给出硬件的数学语言：布尔代数。第 4 章用门搭出加法器等组合积木。第 5 章引入记忆元件，让电路拥有状态。第 6 章讨论时间的纪律：为什么芯片有主频，主频由什么决定。第 7 章用状态机让电路按剧本行事。第 8 章把前面的一切装进硬件描述语言，学会用代码描述电路。第 9 章俯瞰从代码到硅片的完整流程。

各章环环相扣，建议按顺序阅读。若你已有数字电路基础，可以从第 5 章开始，但请至少浏览第 1 章，它奠定了全书的视角。

记号约定

逻辑变量用大写斜体字母（如 A 、 B ）表示；逻辑非记作 $\overline{A}$ ，逻辑与记作 $A \cdot B$ （有时省略点号写作 AB ），逻辑或记作 $A + B$ 。二进制数加下标 2，如 $(1011)_2$ ；不加下标的数默认为十进制。电路图中的逻辑门采用国家标准 GB/T 4728 的矩形符号。

好了，让我们从一个看似天真的问题开始：计算机为什么非要用 0 和 1？

目录

前言	1
本书的写法	1
各章的路线图	1
记号约定	2
1 为什么是 0 和 1: 数字抽象	5
1.1 信息需要物理载体	5
1.2 数字抽象: 用区间表示符号	5
1.3 再生: 数字信号能够自我修复	7
1.4 为什么是二, 不是三、不是十	8
1.5 用比特承载万物	8
2 开关的艺术: 晶体管与逻辑门	11
2.1 晶体管: 受电压控制的开关	11
2.2 第一个逻辑门: CMOS 反相器	12
2.3 与非门与或非门	13
2.4 门符号与抽象的力量	14
2.5 与非门是“万能”的	15
2.6 功耗: 翻转是要付账的	16
3 布尔代数: 硬件的数学语言	18
3.1 三副面孔与一套代数	18
3.2 德摩根定律: 取反的艺术	20
3.3 从真值表到表达式: 最小项	20
3.4 化简: 更少的门, 更快的路	21
4 组合逻辑的积木: 选择与加法	24
4.1 多路选择器: 硬件的 if	24
4.2 加法器: 从一位开始	26
4.3 多位加法: 行波进位	27
4.4 超前进位: 用面积买时间	28
5 记忆的诞生: 锁存器与触发器	30
5.1 反馈: 记忆的种子	30
5.2 SR 锁存器: 可以改写的记忆	30
5.3 D 锁存器: 数据与时机分离	31
5.4 边沿触发: 主从结构与 D 触发器	32
5.5 时钟: 把连续时间切成节拍	33

5.6 亚稳态：山顶上的球	34
6 时间的纪律：建立、保持与流水线	36
6.1 触发器的契约：三个时间参数	36
6.2 建立约束：主频的由来	37
6.3 保持约束：另一半契约	37
6.4 流水线：用并行换节拍	38
7 有限状态机：让电路按剧本行事	41
7.1 状态机的骨架	41
7.2 完整实战：序列检测器	42
7.3 状态编码的门道	43
8 用代码描述硬件：Verilog 与 RTL 思维	45
8.1 描述，而不是指令	45
8.2 模块与组合逻辑	46
8.3 时序逻辑：让时钟说话	47
8.4 阻塞与非阻塞：一字之差，两种电路	47
8.5 状态机进 Verilog	49
8.6 验证初步：测试平台	50
9 从代码到硅片：设计流程全景	52
9.1 鸟瞰：一块芯片的旅程	52
9.2 验证：为什么它是大头	53
9.3 PPA：工程的三角权衡	54
9.4 结语：你已经走到哪里	54
附录 A 思考题参考答案	56
第 1 章	56
第 2 章	56
第 3 章	57
第 4 章	58
第 5 章	58
第 6 章	59
第 7 章	59
第 8 章	60
第 9 章	61

第一章

为什么是 0 和 1：数字抽象

一块现代手机芯片上集成着上百亿个晶体管，每一个都可能出错一点点：温度在变，电压在抖，导线之间在互相干扰。然而芯片算出的每一笔账都分毫不差。让近乎不可靠的物理世界，承载绝对可靠的计算，靠的是全书的第一块基石：数字抽象。本章要回答三个问题：信息如何寄居在电压里？为什么“用区间表示符号”能抵抗噪声？以及，为什么偏偏是二进制？

1.1 信息需要物理载体

信息本身是无形的，它必须寄居在某种物理量上才能被存储和传递：声音寄居在空气的振动里，文字寄居在纸上的墨迹里。在电子系统中，最方便的载体是**电压**，因为电压改变起来极快，又几乎不消耗空间。

最直接的想法是让电压与信息成正比：温度 25.3 度，就用 2.53 V 表示。这种“物理量连续地正比于信息”的表示方法称为**模拟表示**。早期的电话、电视、录音机都是模拟系统。

模拟表示有一个致命的弱点：**噪声会累积，而且无法撤销**。电压在传输和处理中难免被叠加上各种微小的扰动。假设 2.53 V 的信号经过一级放大器后变成了 2.55 V，接收方看到 2.55 V 时无从判断：这是本来就该有的值，还是被噪声污染的结果？信息与噪声混在同一个连续标尺上，没有任何办法把它们分开。于是每经过一级处理，信号就变差一点。你可以想象“复印件的复印件”：复印一次损失一点清晰度，复印十次之后就面目全非。模拟系统做不长处理链条，根源就在这里。

1.2 数字抽象：用区间表示符号

数字系统换了一种根本不同的思路：**不再让电压的精确值携带信息，而是把电压范围划分成几个区间，只用“落在哪个区间”携带信息**。

最常见的划分是两个区间。以早期广泛使用的 5 V 供电的 TTL 电路为例，约定如下：

- 电压低于 0.8 V，一律判读为逻辑 **0**；
- 电压高于 2.0 V，一律判读为逻辑 **1**；
- 两者之间是**禁区**：合法的稳定信号不允许停留在这里。

定义 数字抽象

把连续的物理量（如电压）划分为有限个区间，每个区间代表一个离散符号；电路只关心信号落在哪个区间，不关心它的精确值。这一约定称为数字抽象（digital abstraction）。

这样一来，2.53 V 被噪声扰动成 2.55 V 甚至 2.9 V，都无关紧要：它们统统高于 2.0 V，判读结果都是逻辑 1，信息毫发无损。**噪声只有大到把信号推过区间边界，才会造成错误。**我们主动放弃了电压所能承载的绝大部分细节，换来的是对噪声的免疫力。这笔“用精度换可靠”的交易，是整个数字世界的地基。

工程上还留了一手：对“输出”的要求比对“输入”的要求更严格。仍以 TTL 为例，门电路输出逻辑 1 时保证电压不低于 2.4 V，而接收方只要求高于 2.0 V 就认作 1。中间这 0.4 V 的富余量称为**噪声容限**（noise margin）：信号在路上被噪声侵蚀 0.4 V 以内，接收方照样判读正确。图 1.1 画出了这套电平约定。

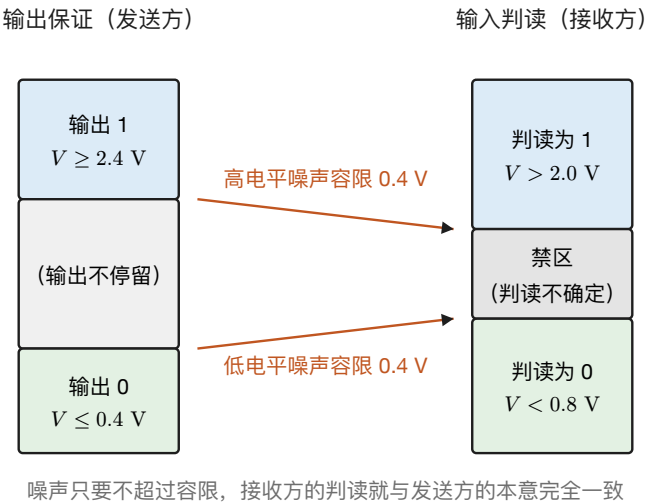


图 1.1 TTL 电平约定与噪声容限。发送方的输出被压在两端，接收方的判读区间更宽，中间的富余量就是抵抗噪声的“护城河”。

例 1.1 计算噪声容限

某逻辑家族规定：输出低电平至多 $V_{OL} = 0.4\text{ V}$ ，输出高电平至少 $V_{OH} = 2.4\text{ V}$ ；输入低于 $V_{IL} = 0.8\text{ V}$ 判为 0，高于 $V_{IH} = 2.0\text{ V}$ 判为 1。求两个方向的噪声容限。

解 考虑最坏情况。发送 0 时，输出最高可达 0.4 V；接收方容忍到 0.8 V。因此低电平噪声容限为

$$NM_L = V_{IL} - V_{OL} = 0.8 - 0.4 = 0.4\text{ V}$$

同理，高电平噪声容限为

$$NM_H = V_{OH} - V_{IH} = 2.4 - 2.0 = 0.4\text{ V}$$

这意味着：只要一路上叠加的噪声不超过 0.4 V，逻辑值就绝不会被误读。

1.3 再生：数字信号能够自我修复

噪声容限解释了数字信号为什么“扛得住”一段路的噪声，但还有更深一层的机制，解释它为什么能扛住任意长的处理链条。

看一个最简单的数字电路：反相器（后面章节会讲它的内部构造）。它的职责是把 0 变成 1、把 1 变成 0。关键在于它对电压的响应曲线，如图 1.2 所示：只要输入落在合法的低区间内，无论是 0.1 V 还是 0.7 V，输出都是几乎不打折扣的标准高电平；反之亦然。

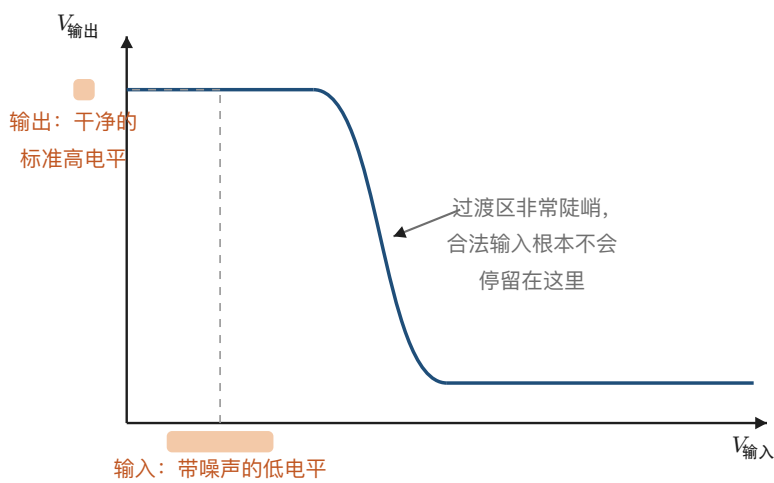


图 1.2 反相器的电压传输特性（示意）。平坦的两端把输入端的噪声“压扁”，输出重新回到标准电平。

请注意图中被着色的两段：输入端一个相当宽的噪声带，经过反相器后被压缩成输出端一个极窄的带。换句话说，每经过一级门电路，信号不是变得更脏，而是被重新整形为干净的标准电平。噪声在每一级都被清零，自然永远无法累积。这个性质称为再生（regeneration）或恢复特性。

本质要点

数字系统可靠性的本质是：每一级电路都重新做一次“非此即彼”的决定，把上一级残留的误差彻底抹掉。模拟系统里噪声乘着信号一路搭便车；数字系统里，噪声每过一站就被请下车。正因如此，数字电路才能串联成千上万级而不失真，芯片的规模才可能无限生长。

1.4 为什么是二，不是三、不是十

数字抽象只要求“有限个区间”，并没有规定必须是两个。历史上确实出现过三进制计算机（例如苏联 1958 年的 Сетунь 机）。那么现代芯片为什么无一例外选择了二进制？主要有两个理由。

第一，**两个区间给出最大的噪声容限**。供电电压的总范围是有限的，划分的区间越多，每个区间就越窄，区间之间的“护城河”也越窄，同样的噪声就更容易把信号推过界。二区间是划分最粗、因而最皮实的方案。

第二，**“开关”是物理上最容易做好的器件**。让一个器件精确地停在十个不同的状态很难，但让它要么完全导通、要么完全截止，却极其容易，而且天然对应两个区间。下一章将看到，晶体管恰好就是这样一个优秀的开关。

付出的代价是表示同一个数需要更多的位数。但下面的数学加油站会说明，这个代价只是一个不大的常数因子，而可靠性的收益却是决定性的。这是一笔非常划算的交易。

数学加油站 进位制与位数的代价

十进制数“逢十进一”，每一位有 10 种符号，第 k 位的“分量”（称为**位权**）是 10^k 。二进制完全同理，只是“逢二进一”，位权是 2^k 。例如

$$(1011)_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 0 + 2 + 1 = 11$$

用 n 位二进制可以表示 2^n 个不同的数（从 0 到 $2^n - 1$ ）。要表示 N 个不同的数，十进制大约需要 $\log_{10} N$ 位，二进制大约需要 $\log_2 N$ 位。根据对数的换底公式

$$\log_2 N = \frac{\log_{10} N}{\log_{10} 2} \approx 3.32 \times \log_{10} N$$

也就是说，二进制的位数只比十进制多约 3.32 倍，与 N 本身多大无关。用一个固定的常数倍长度，换取每一位都极端可靠，这就是二进制的核心理由。

1.5 用比特承载万物

一旦接受了二进制，一个只取 0 或 1 的最小信息单元就有了名字：**比特** (bit, binary digit 的缩写)。比特是数字世界的原子，万物都编码为比特串：

- 无符号整数：直接按位权展开，如 $(1011)_2 = 11$ ；
- 文字：约定一张编码表，例如字母 A 编码为 01000001（ASCII 码 65）；
- 图像：每个像素的红、绿、蓝亮度各用若干位表示；
- 指令：处理器把“做什么操作、操作谁”也编码成比特串，这是第 9 章的伏笔。

稍微不那么直观的是**负数**的表示。现代芯片一律采用**补码** (two's complement)，它的巧妙之处在于：让加法电路不加任何修改就能同时处理正数和负数。理解补码需要一个小小的数学概念：模运算。

数学加油站 模运算：在“钟面”上做算术

钟面上只有 1 到 12。现在是 10 点，过 5 小时是 3 点，因为超过 12 就绕回来： $10 + 5 = 15$ ， $15 - 12 = 3$ 。这种“到头就绕回”的算术称为**模运算**，记作

$$15 \equiv 3 \pmod{12}$$

读作“15 与 3 关于模 12 同余”，意思是两者除以 12 的余数相同。

模运算带来一个有趣的效果：**减法可以伪装成加法**。在钟面上“往回拨 3 格”和“往前拨 9 格”结果完全一样，因为 $-3 \equiv 9 \pmod{12}$ 。也就是说，只要世界是绕圈的，每个减法都有一个加法替身。

n 位二进制寄存器恰好是一个“ 2^n 格的钟面”：最大值再加 1 就绕回 0（最高位的进位被丢弃）。补码正是利用这一点，把“减 N ”替换成“加 $2^n - N$ ”。

例 1.2 8 位补码

在 8 位系统中（钟面共 $2^8 = 256$ 格）， -3 的补码表示是

$$256 - 3 = 253 = (11111101)_2$$

验算 $5 + (-3)$ ：

$$00000101 + 11111101 = 1\ 00000010$$

结果共 9 位，最高位的进位 1 超出寄存器宽度被自然丢弃（这正是“绕回”），剩下 $(00000010)_2 = 2$ 。答案正确，而加法电路对“负数”一无所知，它只是老实地做了一次模 256 的加法。

约定最高位为 1 的一半格子代表负数，8 位补码便可表示 -128 到 $+127$ 。判断正负只需看最高位，这一位因此称为**符号位**。

常见误区

初学者常以为“最高位是符号、剩下几位是数值”，即所谓“原码”直觉。按这种理解， $(11111101)_2$ 应是 -125 ，而它在补码中实际是 -3 。补码不是“符号加数值”，而是“绕圈算术中的替身”： $-N$ 用 $2^n - N$ 顶替。原码方案需要专门的减法电路，还有 $+0$ 和 -0 两个零；补码让同一套加法电路通吃加减，这正是它统一天下的原因。

还要提醒一点： n 位寄存器只有 2^n 个格子，超出范围的结果必然出错，这称为**溢出**（overflow）。例如 8 位补码中 $127 + 1$ 会得到 -128 。溢出不是电路故障，而是“用有限格子模拟无限整数”这一抽象固有的边界。优秀的设计者永远记得自己的数在多宽的钟面上打转。

本章小结

- 信息必须寄居在物理量上；模拟表示让电压的精确值携带信息，噪声因此无法与信息分离，且逐级累积。
- 数字抽象只用“电压落在哪个区间”携带信息，噪声不越界即无害；输出标准比输入标准更严，两者之差就是噪声容限。
- 门电路的陡峭传输特性使信号逐级再生，噪声每过一级被清零，处理链条因此可以无限延长。
- 选择二进制，是因为两个区间的噪声容限最大，且“开关”是最容易做好的器件；位数的代价只是常数倍（约 3.32 倍于十进制）。
- 比特可以编码万物；负数采用补码，本质是模 2^n 算术中“减法变加法”，同一套加法电路通吃加减；有限位宽必有溢出。

思考题（答案见附录 A）

- 1.1 模拟电视信号变弱时，画面上雪花噪点逐渐增多、缓慢变糟；数字电视信号变弱时，画面长期保持完美，然后突然马赛克甚至黑屏。请用本章的概念解释这种“悬崖效应”。
- 1.2 某 3.3 V 逻辑家族规定 $V_{OL} = 0.2 \text{ V}$ ， $V_{OH} = 3.1 \text{ V}$ ， $V_{IL} = 0.9 \text{ V}$ ， $V_{IH} = 2.4 \text{ V}$ 。求两个噪声容限。若想进一步增大噪声容限，可以调整哪些参数？会付出什么代价？
- 1.3 写出 -1 在 8 位补码中的表示，并解释为什么“全 1”这个码字表示 -1 而不是别的数。再说明 8 位补码为什么能表示 -128 却不能表示 $+128$ 。
- 1.4 同样把 0 V 到 3.3 V 划分区间，三进制需要三个区间。假设区间之间需要留出与区间同宽的禁区，粗略比较二进制与三进制的噪声容限。三进制每一位能携带 $\log_2 3 \approx 1.58$ 比特信息，你认为这笔交易划算吗？

第二章

开关的艺术：晶体管与逻辑门

上一章我们约定了信息的表示法：电压区间代表 0 和 1。本章要造出加工这种信息的最小机器。你将看到，芯片里的一切奇迹都始于一个朴素的器件：受电压控制的开关。用两种互补的开关，我们能搭出“非”“与非”“或非”这些最基本的逻辑门；而只要有了其中一种门，理论上就能搭出任何数字系统。

2.1 晶体管：受电压控制的开关

数字芯片的基本器件是 **MOS 场效应晶体管** (MOSFET, 以下简称 MOS 管)。它有三个端子：**源极** (source)、**漏极** (drain) 和 **栅极** (gate)。源极与漏极之间是一条可以导电的通道；栅极隔着一层极薄的绝缘氧化物覆在通道上方，像一位隔着玻璃发号施令的指挥。

它的工作原理一句话即可说清：**栅极上的电压产生电场，电场决定通道里有没有足够的载流子，从而决定源漏之间是导通还是断开**。栅极与通道之间隔着绝缘层，几乎没有电流流入栅极，所以它是“用电压指挥、不费电流”的开关。“场效应”这个名字，说的就是电场隔空控制电流。

MOS 管有互补的两种。在数字电路的语境下，它们的行为可以概括为一对简明的规则：

- **NMOS 管**：栅极为高电平 (1) 时导通，低电平 (0) 时截止。它擅长把输出拉到低电平，习惯上安放在电路靠近地 (GND) 的一侧。
- **PMOS 管**：恰好相反，栅极为低电平 (0) 时导通，高电平 (1) 时截止。它擅长把输出拉到高电平，安放在靠近电源 (V_{DD}) 的一侧。

图 2.1 给出两种管子的电路符号和对应的开关模型。PMOS 栅极上的小圆圈表示“低电平有效”，这个圆圈记号后面还会反复出现。

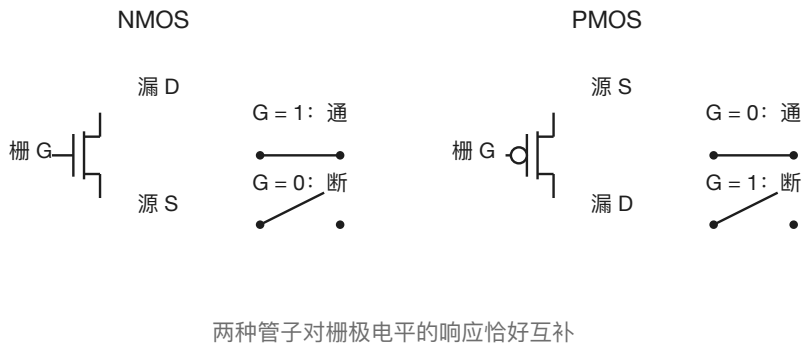


图 2.1 MOS 管的符号与开关模型。PMOS 栅极上的小圆圈表示“低电平导通”。

真实的晶体管当然比“开关”复杂：导通时有电阻，翻转需要时间，截止时也有微弱的漏电流。但数字设计的美妙之处在于，这些细节在多数时候都可以先放进抽象的口袋里，用“开或关”来思考。需要时（例如第 6 章讨论延迟时），我们再把口袋打开。

2.2 第一个逻辑门：CMOS 反相器

用一个 PMOS 和一个 NMOS，就能搭出数字世界的“第一块积木”：反相器（非门）。电路如图 2.2 所示：两管的栅极连在一起作为输入 A ，漏极连在一起作为输出 Y ；PMOS 的源极接电源 V_{DD} ，NMOS 的源极接地。

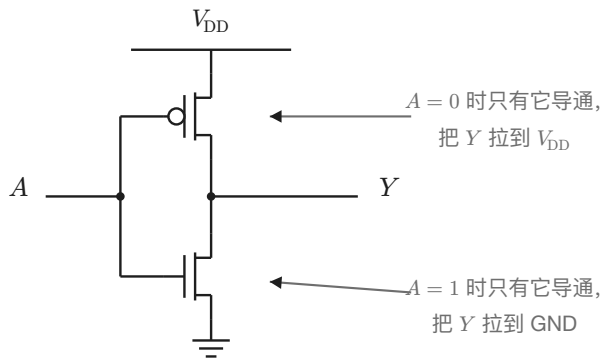


图 2.2 CMOS 反相器。上方 PMOS 负责“拉高”，下方 NMOS 负责“拉低”，任何时刻恰有一个导通。

逐种输入分析它的行为：

输入 A	PMOS（上）	NMOS（下）	输出 Y
0	导通	截止	接通 V_{DD} ，输出 1
1	截止	导通	接通 GND，输出 0

无论输入是 0 还是 1，**两个管子总是恰好一个导通、一个截止**。这带来两个宝贵的性质。

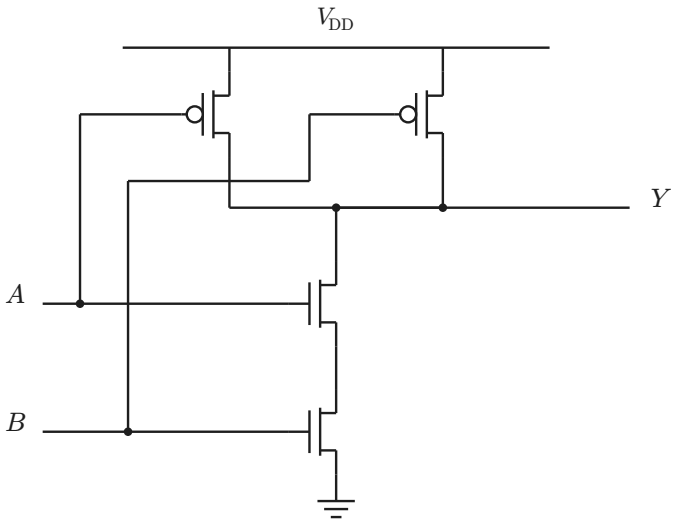
其一，输出在任何时刻都被牢牢“驾驶”着：要么被拉向电源，要么被拉向地，绝不悬空。输出总能给出干净的标准电平，上一章所说的再生特性由此而来。

其二，从电源到地永远不存在一条同时导通的直流通路。电流只在输出翻转的瞬间流动（给负载电容充电或放电），电路静止时几乎分文不费。**静态功耗近似为零**，是这种电路结构最著名的优点。

这种“PMOS 在上拉、NMOS 在下拉、两者互补”的结构称为 **CMOS** (Complementary MOS, 互补式 MOS)。上方所有 PMOS 构成**上拉网络**，下方所有 NMOS 构成**下拉网络**；设计的要义是让两个网络的导通条件恰好互补：任何输入组合下，恰有一个网络导通。现代数字芯片几乎全部采用 CMOS 结构。

2.3 与非门与或非门

反相器只有一个输入。把管子的数目加倍，就能得到两输入的门。图 2.3 是**与非门** (NAND) 的电路：下拉网络是两个 NMOS **串联**，上拉网络是两个 PMOS **并联**。



PMOS 并联：任一输入为 0 即拉高 NMOS 串联：两输入全为 1 才拉低

图 2.3 CMOS 与非门。只有 A、B 同时为 1，串联的下拉通路才导通，输出才为 0。

分析它的逻辑：只有 A 和 B **同时** 为 1，两个串联的 NMOS 才会同时导通，把输出拉到 0；其余任何情形下，至少有一个 PMOS 导通，把输出拉到 1。写成真值表（枚举所有输入组合及其输出的表格）如下：

A	B	$Y = \overline{A \cdot B}$
0	0	1
0	1	1
1	0	1
1	1	0

这正是“与”运算再取反，故名**与非**。请留意上下两个网络的巧妙对应：下拉网络串联（同时满足才通），上拉网络就并联（任一满足即通），两者的导通条件天然互补。把这个对应关系反过来用，就得到**或非门**（NOR）。

例 2.1 设计 CMOS 或非门

或非的定义是 $Y = \overline{A + B}$ ：任一输入为 1，输出即为 0。
解 “任一输入为 1 就拉低”，说明下拉网络应当是两个 NMOS **并联**；按互补对应，上拉网络就是两个 PMOS **串联**（只有 A、B 全为 0，输出才被拉高）。读者可以自行画出电路并逐行验证真值表：它与与非门恰好是“串并调”的镜像。

一个值得玩味的现象：用 CMOS 直接搭出来的门天然都带“非”（反相器、与非、或非），想要不带非的与门，反而要用与非门再接一个反相器。原因就藏在开关的性格里：NMOS 导通时拉出的是 0，PMOS 导通时拉出的是 1，而它们又分别被 1 和 0 触发，一进一出必然翻转一次。带“非”的门才是 CMOS 的母语。

2.4 门符号与抽象的力量

从本节起，我们不再画晶体管，而是用标准符号表示逻辑门。图 2.4 列出了本书用到的六种门（采用国家标准的矩形符号：方框中的“&”表示与，“≥ 1”表示或，“=1”表示异或，输出端小圆圈表示取反）。

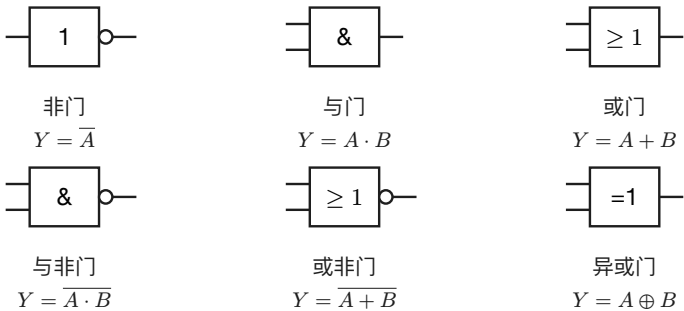


图 2.4 本书使用的逻辑门符号。异或（XOR）在两输入不同时输出 1，第 4 章的加法器将大量使用它。

其中**异或门** (XOR, 记号 $\oplus$) 值得单独介绍: $A \oplus B$ 在 A 、 B **不同时** 为 1、相同时为 0。它可以由前面的门组合而成 ($A \oplus B = A\bar{B} + \bar{A}B$), 因使用频繁而获得了自己的符号。

请注意刚才发生的事情: 我们把四个或六个晶体管封装成一个符号, 然后**忘掉晶体管**。今后使用与非门时, 只需记得它的真值表和延迟, 不必关心内部的串并结构。这就是**抽象分层**: 每一层只依赖下一层的**对外承诺** (学术上称为**契约**), 而不依赖其实现细节。

本质要点

数字芯片是一座抽象的高塔: 晶体管 $\rightarrow$ 逻辑门 $\rightarrow$ 功能模块 (加法器、寄存器) $\rightarrow$ 子系统 (处理器核) $\rightarrow$ 芯片。每一层都把下层的复杂性封装成几条简单的承诺, 供上层放心取用。人脑同时只能把握几个对象, 正是这座塔让几十个人的团队驾驭百亿晶体管成为可能。本书后面的每一章, 本质上都是在这座塔上再砌一层。

2.5 与非门是“万能”的

一个自然的问题: 这些门里, 哪些是必不可少的? 答案出人意料地激进: **只要与非门一种, 就够了**。

数学加油站 功能完备性

若一个门 (或一组门) 通过反复组合能实现**任意**真值表, 就称它是**功能完备**的 (functionally complete)。

可以先承认一个事实 (第 3 章将给出一般方法): {非、与、或} 三件套是功能完备的, 任何真值表都能翻译成由这三种运算组成的表达式。因此, 只要证明某个门能造出这三件套, 它就继承了完备性。

与非门恰好能造出三件套。第一步造“非”: 把与非门的两个输入接在一起, 得到 $\overline{A \cdot A} = \bar{A}$, 这就是反相器。第二步造“与”: 与非之后再接一个刚造好的“非”, 负负得正, $\overline{\overline{A \cdot B}} = A \cdot B$ 。第三步造“或”, 需要借助一条代数恒等式 (德摩根定律, 第 3 章证明): $A + B = \overline{\bar{A} \cdot \bar{B}}$, 即“先把两个输入各自取反, 再与非”。

例 2.2 只用与非门实现或门

按上面的思路: 用两个“输入并接”的与非门分别得到 $\bar{A}$ 和 $\bar{B}$, 再把它们送入第三个与非门:

$$Y = \overline{\bar{A} \cdot \bar{B}} = A + B$$

共用三个与非门。验证一行真值表: 当 $A = 0, B = 1$ 时, $\bar{A} = 1, \bar{B} = 0$, 与非输出 $\overline{1 \cdot 0} = 1$, 与“或”的定义一致。其余三行留给读者。

功能完备性有深刻的工程意义：芯片制造厂只需把少数几种基本门做到极致（速度、功耗、良率），就足以支撑一切数字产品。事实上，芯片设计流程中的“标准单元库”（第9章）正是这一思想的产物：任何复杂设计最终都被翻译成几百种预先设计好的小单元的排列组合。

2.6 功耗：翻转是要付账的

CMOS 静止时几乎不耗电，那电都耗在哪里？耗在**翻转**上。输出每次从 0 变 1，电源就要给输出节点的电容充一次电；从 1 变 0，这些电荷又被泄放到地。每次充放电消耗的能量约为 CV^2 （ C 是节点电容， V 是电源电压）。若时钟频率为 f ，且平均每个时钟周期有比例为 α 的节点翻转，则动态功耗为

$$P \approx \alpha \cdot C \cdot V^2 \cdot f$$

这个公式解释了芯片工业的许多现象。为什么降低电压是最诱人的省电手段？因为 V 带平方，电压降到 0.7 倍，动态功耗就降到约一半。为什么手机芯片在空闲时要关闭时钟（clock gating）？因为 f 或 α 归零，动态功耗随之归零。为什么本世纪初处理器主频冲到约 4 GHz 后停滞不前，转而走向多核？因为功耗随频率与电压水涨船高，散热撞上了物理极限，业界只好用“更多而不是更快”的核来换取性能。

需要补一句：当晶体管缩小到纳米尺度后，截止的管子也有不可忽略的漏电流，**静态功耗**卷土重来，成为先进工艺的重要挑战。不过对本书的入门视角而言，记住“翻转付账、静止近乎免费”这条主线就够了。

本章小结

- MOS 管是受电压控制的开关：NMOS 栅极为 1 导通、擅长拉低；PMOS 栅极为 0 导通、擅长拉高。
- CMOS 结构用互补的上拉、下拉网络保证：输出任何时刻都被驾驶，且无静态直流通路；下拉串联对应上拉并联，反之亦然。
- 反相器、与非、或非是 CMOS 的“母语”；与非门功能完备，一种门即可搭出万物。
- 门符号是一次抽象封装：上层只依赖真值表与延迟等承诺，不依赖晶体管细节。抽象分层是驾驭百亿晶体管的根本方法。
- 动态功耗 $P \approx \alpha CV^2 f$ ：翻转付账，静止近乎免费；电压的平方项使降压成为最有效的节能手段。

思考题（答案见附录 A）

2.1 仿照图 2.3，画出（或用文字描述）三输入与非门的晶体管结构：下拉网络和上拉网络各由哪些管子、按什么方式连接？共需多少个晶体管？

2.2 如果 CMOS 反相器的输入端悬空，或长期停在 $V_{DD}/2$ 附近的中间电压，会发生什么？请从上下两管的导通状态和功耗两个角度分析，并解释为什么数字设计严禁把稳定信号留在禁区。

2.3 异或可写作 $A \oplus B = \overline{A}B + A\overline{B}$ 。请只用与非门实现异或门，并数一数最少用了几个（提示：四个即可；先试着把表达式变形成只含与非的形式）。

2.4 某芯片把电源电压从 1.0 V 降到 0.7 V，同时把频率从 3 GHz 降到 2 GHz。假设 α 与 C 不变，动态功耗变为原来的百分之多少？这解释了移动芯片的哪种常见策略？

第三章

布尔代数：硬件的数学语言

上一章我们有了积木，这一章要学会积木的语言。逻辑电路有三副面孔：真值表、代数表达式、电路图。它们描述的是同一个东西，却各有所长：真值表适合罗列需求，表达式适合推导变形，电路图对应真实的门。数字设计的日常，就是在三者之间自如穿梭：把需求写成真值表，把真值表翻译成表达式，把表达式化简到最省，再落成电路。本章讲授这门手艺，它的数学名字叫布尔代数。

3.1 三副面孔与一套代数

布尔代数（Boolean algebra）是十九世纪数学家布尔为“思维的规律”建立的代数：变量只取 0 和 1 两个值，基本运算只有三种，即非 ($\overline{A}$)、与 ($A \cdot B$)、或 ($A + B$)。一个世纪后，香农在他著名的硕士论文中指出：开关电路恰好服从这套代数。从此，设计电路的问题第一次变成了可以纸上推演的数学问题。

图 3.1 概括了本章的全局：三种表示互相等价、互相转换。

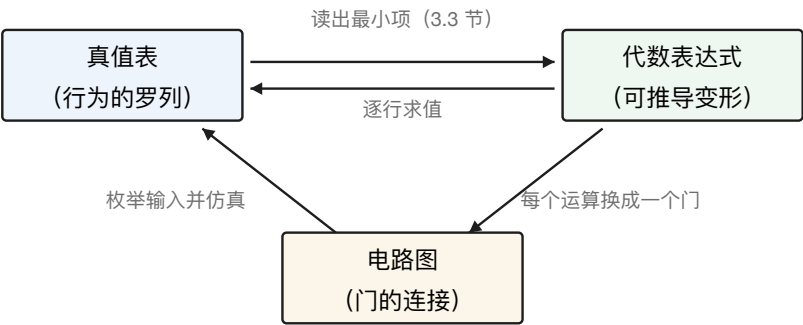


图 3.1 逻辑函数的三种表示及其转换路径。三者携带同样的信息，服务于不同的用途。

布尔代数的常用定律汇总如下。多数与普通代数似曾相识，但有两条例外值得特别留意。

定律	“或”形式	“与”形式
交换律	$A + B = B + A$	$AB = BA$
结合律	$(A + B) + C = A + (B + C)$	$(AB)C = A(BC)$
分配律	$A(B + C) = AB + AC$	$A + BC = (A + B)(A + C)$
同一律	$A + 0 = A$	$A \cdot 1 = A$
支配律	$A + 1 = 1$	$A \cdot 0 = 0$
幂等律	$A + A = A$	$A \cdot A = A$
互补律	$A + \overline{A} = 1$	$A \cdot \overline{A} = 0$
吸收律	$A + AB = A$	$A(A + B) = A$
德摩根定律	$\overline{A + B} = \overline{A} \cdot \overline{B}$	$\overline{AB} = \overline{A} + \overline{B}$

第一条例外是分配律的“与”形式： $A + BC = (A + B)(A + C)$ 。在普通代数里“加法对乘法分配”并不成立，在布尔代数里却成立。第二条例外是幂等律： $A + A = A$ ，没有“2A”这种东西，因为0和1的世界里不存在“二”。

还请观察表格的左右两列：把任何一条定律中的“或”与“与”互换、“0”与“1”互换，就得到另一条成立的定律。这个漂亮的性质称为**对偶原理**。它不是巧合，第2章CMOS电路里“下拉串联对应上拉并联”的镜像结构，正是对偶原理在晶体管层面的倒影。

如何证明这些定律？布尔代数给了我们一件普通代数没有的奢侈品：**变量的取值只有有限种，可以穷举**。把等式两边对所有输入组合各算一遍，若每行都相等，等式即告成立。这种方法称为**完备归纳法**，真值表就是它的工作底稿。

例 3.1 两种方法证明吸收律 $A + AB = A$

方法一：真值表穷举。两个变量共 $2^2 = 4$ 种组合：

A	B	AB	A + AB
0	0	0	0
0	1	0	0
1	0	0	1
1	1	1	1

末列与 A 列处处相同，证毕。

方法二：代数推导。

$$A + AB = A \cdot 1 + AB = A(1 + B) = A \cdot 1 = A$$

依次用了同一律、分配律和支配律。

吸收律的直觉：若 A 为真， $A + AB$ 已经为真，AB 帮不上忙；若 A 为假，AB 也必为假。所以 AB 这一项从头到尾说了不算，可以整项扔掉。化简电路时它对应“拆掉一个多余的门”。

3.2 德摩根定律：取反的艺术

九条定律中，**德摩根定律**值得单独一节，因为它是数字设计中使用最频繁、也最容易用错的一条：

$$\overline{A + B} = \overline{A} \cdot \overline{B}, \quad \overline{A \cdot B} = \overline{A} + \overline{B}$$

用日常语言读第一条：“并非（A 或 B）”等价于“非 A **且**非 B”。“我没去过北京或上海”意思是“我没去过北京，**而且**没去过上海”。取反号越过括号时，里面的“或”要翻成“与”。这正是初学者最常犯的错误：把 $\overline{A + B}$ 误写成 $\overline{A} + \overline{B}$ 。用一行反例即可自查： $A = 1, B = 0$ 时，左边 $\overline{1 + 0} = 0$ ，右边 $\overline{1} + \overline{0} = 0 + 1 = 1$ ，两边并不相等。

对硬件工程师来说，德摩根定律是一台“门的变形机”。把 $\overline{AB} = \overline{A} + \overline{B}$ 从右往左读：一个与非门，等价于“输入先各自取反，再送进或门”。第 2 章用与非门搭或门的戏法，代数依据正是它。工程上有个形象的说法叫**推气泡**：把门输出端的取反小圆圈“推”到输入端去，同时把与换成或（或反之），电路功能不变。熟练的设计者靠这一手在与非、或非、与、或之间自由改写电路。

3.3 从真值表到表达式：最小项

现在兑现第 2 章的承诺：**任何真值表都能机械地翻译成表达式**。方法简单得出奇，我们用一个实例来演示。

例 3.2 多数表决器

三人委员会表决，赞成过半（至少两人）则议案通过。设三人的投票为 $A、B、C$ （1 表示赞成），求“通过”信号 F 的表达式。
第一步，列真值表：

A	B	C	F	对应最小项
0	0	0	0	
0	0	1	0	
0	1	0	0	
0	1	1	1	$\overline{A}BC$
1	0	0	0	
1	0	1	1	$A\overline{B}C$
1	1	0	1	$AB\overline{C}$
1	1	1	1	ABC

第二步，盯住输出为 1 的行。每一行对应输入的一种**精确组合**，可以写成一个“与”项：变量在该行取 1 就照写，取 0 就写它的非。例如第四行 $(A, B, C) = (0, 1, 1)$ 对应乘积项 $\overline{A}BC$ ：只有输入恰好等于这个组合时，该项才等于 1。这样的项称为**最小项**（minterm）。

第三步，把这些最小项“或”起来：

$$F = \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC$$

任何一个“该输出 1”的组合出现时，恰有一个最小项变成 1， F 随之为 1；其余组合下所有最小项都是 0。这个表达式忠实复刻了真值表。

这种“输出为 1 的行各写一个最小项，再全部或起来”的形式，称为**积之和**（Sum of Products, SOP）形式。它给出了一个深刻的结论：**任何真值表都能只用非、与、或三种运算写出来**。第 2 章说“{非、与、或}功能完备”，证明就在这里，因为这个翻译流程对任何真值表都行得通。

3.4 化简：更少的门，更快的路

直接按最小项搭电路虽然正确，却往往铺张。上例需要四个三输入与门加一个四输入或门；而我们马上会看到，同样的功能三个两输入与门就够了。**化简**不是洁癖：门更少意味着面积更小、功耗更低，级数更浅往往还意味着速度更快（第 6 章将看到深度如何决定频率）。

先做代数化简。关键观察是：**两个只差一个变量的最小项可以合并，消去那个变量**，例如 $AB\overline{C} + ABC = AB(\overline{C} + C) = AB$ 。利用幂等律把 ABC 复制三份（ $X + X = X$ ，多写不算错），与三个伙伴分别配对：

$$\begin{aligned} F &= \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC \\ &= (\overline{A}BC + ABC) + (A\overline{B}C + ABC) + (AB\overline{C} + ABC) \\ &= BC + AC + AB \end{aligned}$$

结果一目了然：三对投票中任意一对同时赞成，议案即通过。这正是“多数”的另一种说法，化简后的表达式反而更贴近本意。电路如图 3.2 所示。

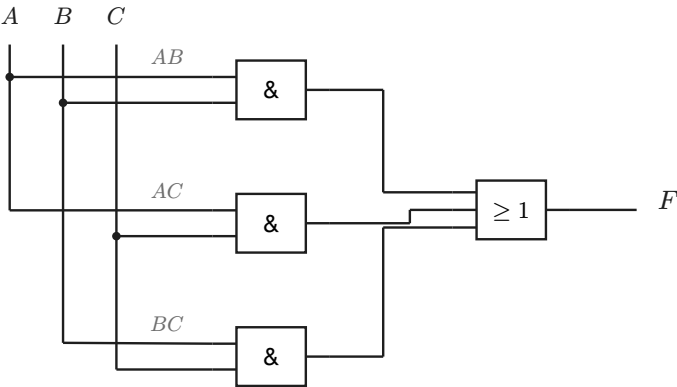


图 3.2 化简后的多数表决器： $F = AB + AC + BC$ 。竖线为输入引线，交叉处不打点表示不相连。

代数化简依赖灵感：往哪个方向变形、把哪一项复制几份，都要靠观察。对三四个变量的函数，有一种把“找可合并项”变成“看图找相邻”的图形化方法，几乎不需要灵感，这就是卡诺图（Karnaugh map）。它的原理建立在一个小小的编码技巧上。

数学加油站 格雷码：只变一位的排序

两个最小项能合并的条件，是它们恰好只有一个变量不同，例如 $ABC\bar{C}$ 与 ABC 。我们把这样的两个输入组合称为相邻的。

普通二进制计数顺序 00、01、10、11 有个讨厌的地方：从 01 到 10 要同时变两位，它们并不相邻。格雷码（Gray code）重新排列顺序为

$$00 \rightarrow 01 \rightarrow 11 \rightarrow 10$$

任何前后两项（包括从末尾绕回开头）都只差一位。按这个顺序给表格的行列编号，“代数上相邻”就变成了“几何上挨着”，肉眼可以直接看出哪些项能合并。这正是卡诺图的行列不按 00、01、10、11 而按 00、01、11、10 排列的原因。

卡诺图就是把真值表重新排版：每个格子放一个最小项的输出值，行列按格雷码排序，使得所有几何相邻（含首尾环绕）的格子都代数相邻。化简的动作变成：把值为 1 的相邻格子圈成矩形块，每块的格子数必须是 2 的幂（1、2、4、8）；每个块对应一个消去了若干变量的乘积项；目标是用尽量少、尽量大的块盖住所有的 1。

例 3.3 用卡诺图化简多数表决器

把例 3.2 的真值表装进卡诺图（行是 A ，列是 BC ，注意列序是格雷码），四个 1 恰好组成三个两格块，见图 3.3。

读块的规则：看块内哪些变量始终不变，不变的写进乘积项，变化的消去。蓝色竖块覆盖 $(A,B,C) = (0,1,1)$ 与 $(1,1,1)$ 两格： A 在块内既取 0 又取 1，被消去； $B、C$ 恒为 1，故得乘积项 BC 。同理绿块给出 AC ，橙块给出 AB 。三块相或：

$$F = AB + AC + BC$$

与代数化简的结果一致，但过程只是“看图圈块”。注意 $(1,1,1)$ 这个 1 被三个块共用，这是幂等律（ $X + X = X$ ）在图上的体现：一个 1 可以尽情重复使用。

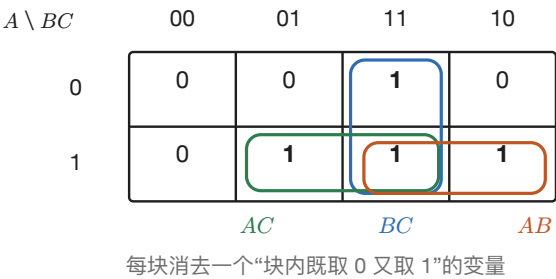


图 3.3 多数表决器的卡诺图。三个两格块分别给出 $BC、AC、AB$ ，重叠处的 1 可被多个块共用。

最后补充一个实战中常见的便利：有些输入组合**根本不会发生**，或者发生了也没人在乎输出是什么。例如用 4 位二进制表示十进制数字 0 到 9 时，1010 到 1111 这六个组合永远不会出现。这类格子称为**无关项**（don't care），在卡诺图中记作“x”，圈块时可以随意当作 1 或 0，哪种能让块更大就当哪种。这是免费的化简筹码，不用白不用。

本章小结

- 布尔代数的变量只取 0 与 1，基本运算是非、与、或；真值表、表达式、电路图三种表示等价互通。
- 定律与普通代数大体相似，但 $A + BC = (A + B)(A + C)$ 与 $A + A = A$ 是布尔特有的；每条定律都有对偶形式。由于取值有限，一切等式都可用真值表穷举证明。
- 德摩根定律是取反跨越括号的规则：与变或、或变与。它支撑了与非、或非门之间的电路改写（推气泡）。
- 任何真值表都可翻译为最小项之和（SOP），这证明了{非、与、或}的功能完备性。
- 化简的代数本质是合并只差一个变量的项；卡诺图用格雷码排版，把“找可合并项”变成“看图圈块”；无关项是免费的化简筹码。

思考题（答案见附录 A）

- 3.1 用真值表证明布尔特有的分配律 $A + BC = (A + B)(A + C)$ 。再用代数方法从右边推到左边（提示：先把右边乘开，然后反复使用幂等律与吸收律）。
- 3.2 利用德摩根定律，把 $\overline{AB + C}$ 改写成只对单个变量取反的形式。
- 3.3 某函数 $F(A, B, C, D)$ 在最小项编号 0、2、5、7、8、10、13、15 处取 1（编号即 $ABCD$ 依次排成的二进制数的十进制值）。画出 4 变量卡诺图并化简。提示：卡诺图的上下边、左右边都首尾相接，四个角互为相邻。
- 3.4 把多个异或门串成链，得 $Y = A_1 \oplus A_2 \oplus \dots \oplus A_n$ 。请说明： $Y = 1$ 当且仅当输入中取 1 的个数为奇数。这个电路在通信和存储系统中被称为什么？有什么用途？

第四章

组合逻辑的积木：选择与加法

掌握了门和代数，就可以搭建真正有用的部件了。本章挑选两类最有代表性的积木：会“选择”的多路选择器，和会“算数”的加法器。选出它们不只因为常用，更因为各自藏着一个深刻的主题：多路选择器背后是“任何函数都能用查表实现”的思想，今天价值百亿美元的 FPGA 产业就建立在它上面；加法器背后是“进位像波浪一样传播”的时间问题，它将把我们引向数字设计的下一个主战场：速度。

在动手之前，先给这一大类电路起个正式的名字。

定义 组合逻辑

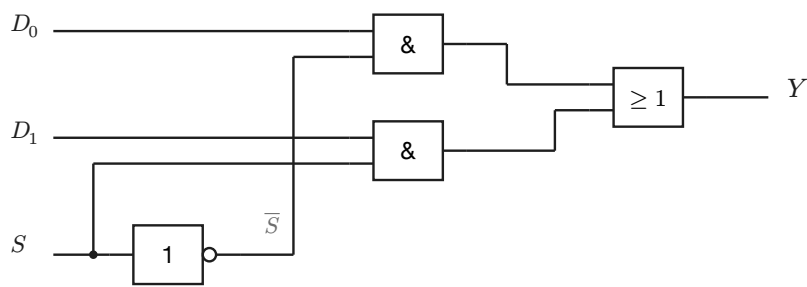
输出只由当前输入决定的电路称为组合逻辑 (combinational logic)。给定输入，输出唯一确定，与历史无关；电路中不存在从输出绕回输入的反馈环。前两章的所有门电路，以及本章的全部积木，都是组合逻辑。与之相对、拥有记忆的电路称为时序逻辑，那是第 5 章的主题。

4.1 多路选择器：硬件的 if

多路选择器 (multiplexer, 简称 mux) 做的事情用一句话就能说完：从多路输入中挑一路送到输出。最小的版本是二选一：数据输入 D_0 、 D_1 ，选择端 S ； $S = 0$ 时输出 D_0 ， $S = 1$ 时输出 D_1 。用软件的话说，它就是硬件里的 if：

$$Y = \overline{S}D_0 + SD_1$$

读一读这个表达式： $S = 0$ 时前一项“放行” D_0 、后一项被压成 0； $S = 1$ 时恰好相反。两个与门是闸门，或门把两路汇合，如图 4.1 所示。



$S = 0$ 时上闸门放行 D_0 ； $S = 1$ 时下闸门放行 D_1

图 4.1 二选一多路选择器： $Y = \overline{S}D_0 + SD_1$ 。与门当闸门，或门做汇合。

推广是自然的：四选一用 2 位选择端，八选一用 3 位， 2^n 选一用 n 位。多路选择器无处不在：处理器里“这条指令的结果来自加法器还是存储器”，总线上“这一拍轮到谁发言”，本质上全是 mux 在做选择。

不过 mux 还有一个更出人意料的本领：**它能实现任何逻辑函数**。这个本领来自一条简洁的数学恒等式。

数学加油站 香农展开：把函数拆成两半

任何布尔函数 F 都可以按其中一个变量 x 拆开：

$$F = \overline{x} \cdot F|_{x=0} + x \cdot F|_{x=1}$$

其中 $F|_{x=0}$ 表示“把 x 强行代成 0 后剩下的函数”（称为 F 对 x 的限制）， $F|_{x=1}$ 同理。道理直白： x 要么是 0 要么是 1；是 0 时 F 表现得像 $F|_{x=0}$ ，是 1 时表现得像 $F|_{x=1}$ 。两种情形分别由 $\overline{x}$ 、 x 把守，互斥地拼成完整的 F 。

对照 mux 的表达式 $Y = \overline{S}D_0 + SD_1$ ，形状一模一样：**mux 就是香农展开的电路化身**。把 x 接到选择端，两个限制函数接到数据端，就实现了 F 。对限制函数继续展开，直到只剩常数 0 或 1，任何函数便化成了一棵 mux 树。

例 4.1 用四选一 mux 实现多数表决器

仍取第 3 章的多数函数 $F = AB + AC + BC$ 。把 A 、 B 接到四选一 mux 的两个选择端，对四种取值分别求限制函数：

AB 取值	此时 F 化简为	对应数据端应接
00	$F = 0 + 0 + 0 = 0$	常数 0
01	$F = 0 + 0 + C = C$	信号 C
10	$F = 0 + C + 0 = C$	信号 C
11	$F = 1 + C + C = 1$	常数 1

于是四个数据端依次接 0、 C 、 C 、1，不再需要任何门，一个 mux 就实现了三变量函数。请自行代入几组输入验证。

沿着这个思路再往前一步：把 n 个输入全部接到选择端，数据端全部接常数，mux 就成了一张“查函数值的表”。数据端的 2^n 个常数正是真值表的输出列；换一组常数，就换了一个函数。这样的结构称为**查找表** (Look-Up Table, LUT)。可编程逻辑器件 **FPGA** 的基本单元正是查找表加触发器（触发器见第 5 章）：所谓“把电路烧进 FPGA”，本质上只是把各个查找表该装的真值表写进去。一个思想撑起一个产业，就是这个例子。

4.2 加法器：从一位开始

加法是数字系统中最频繁的算术操作，也是理解“运算电路”的最佳入口。按照分而治之的思路，先解决一位加一位，再把解决方案复制串联。

一位加一位，输入 A 、 B ，结果需要两位：本位和 S 与进位 C （因为 $1 + 1 = (10)_2$ ，本位 0，向上进 1）。列真值表：

A	B	进位 C	本位和 S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

两个输出都是熟面孔： $S = A \oplus B$ （不同才为 1）， $C = AB$ （都为 1 才进位）。一个异或门加一个与门就完成了，这个小电路称为**半加器**（图 4.2 左）。

它为什么只算“半个”？因为多位加法中，每一位除了加 A 、 B ，还要加**右边邻居送来的进位** C_{in} 。能完成“三个一位数相加”的电路才算完整，称为**全加器**：

A	B	C_{in}	C_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

观察两个输出列，都有漂亮的结构。 S 列：三个输入中有**奇数个 1**时为 1，这正是三输入异或 $S = A \oplus B \oplus C_{in}$ （即第 3 章思考题 3.4 的奇偶函数）。 C_{out} 列：三个输入中**至少两个**为 1 时为 1，这不就是多数表决器吗！第 3 章的例题在此华丽返场：

$$S = A \oplus B \oplus C_{in}, \quad C_{out} = AB + AC_{in} + BC_{in}$$

工程上还常用另一种等价形式，把全加器拼装成“两个半加器加一个或门”，如图 4.2 右：先算 A 加 B ，再把所得本位和与 C_{in} 相加；两步中任何一步冒出进位， C_{out} 即为 1。两步不会同时冒进位，读者不妨想想为什么。

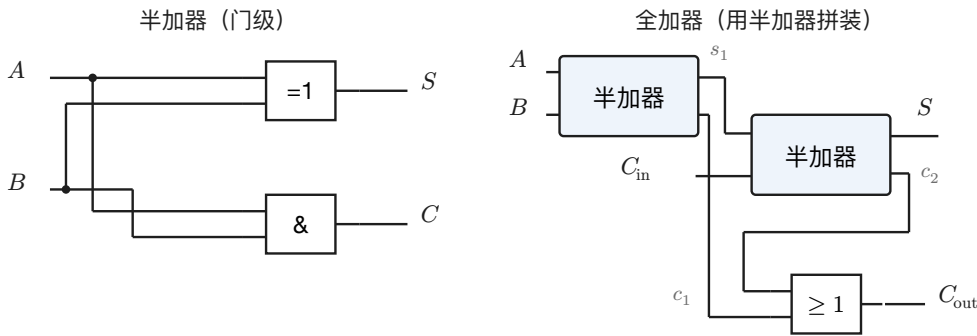


图 4.2 左：半加器， $S = A \oplus B$ ， $C = AB$ 。右：全加器由两个半加器与一个或门拼成。

4.3 多位加法：行波进位

有了全加器，多位加法水到渠成：每一位放一个全加器，让进位像接力棒一样从最低位传到最高位，即第 i 位的 C_{out} 接到第 $i + 1$ 位的 C_{in} 。图 4.3 是 4 位版本，这个结构叫**行波进位加法器**（ripple-carry adder），因为进位像水波一样逐位荡过去。

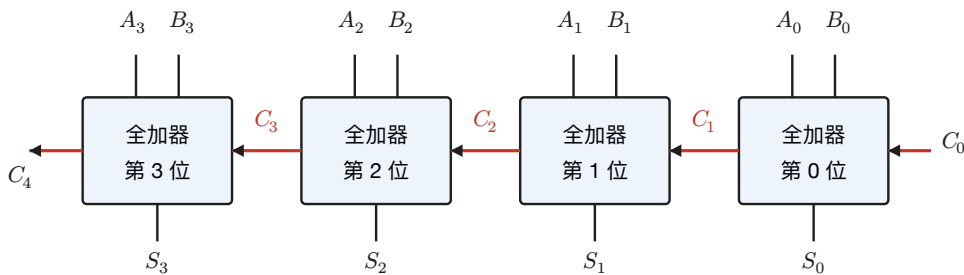


图 4.3 4 位行波进位加法器。红色进位链从第 0 位逐级荡到第 3 位，是整个电路最慢的路径。

行波进位加法器结构规整、面积节省，但藏着一个时间上的隐患。想一想 $0001 + 1111$ ：第 0 位产生进位，推动第 1 位产生进位，再推动第 2 位、第 3 位。**高位在低位的进位到齐之前，给出的都是暂时值**；必须等波浪传完，结果才算数。电路不会“出错”，它只是需要时间。

这引出一个贯穿全书后半部分的核心概念。信号穿过每个门都需要一小段时间，称为**门延迟**（现代工艺中约为几皮秒到几十皮秒；1 皮秒 = 10^{-12} 秒）。一个组合电路从输入变化到输出稳定所需的最长时间，取决于**从输入到输出延迟总和最大的那条路径**，它被称为**关键路径**（critical path）。在行波加法器中，关键路径正是那条红色的进位链：从最低位出发，穿过全部 n 级进位逻辑，抵达最高位。位宽翻倍，延迟也随之翻倍。

例 4.2 估算行波加法器的速度

设每级全加器从 C_{in} 到 C_{out} 的延迟为 2 个门延迟，每个门延迟 30 ps。求 32 位行波加法器进位链的延迟。

解 进位链共 32 级：

$$32 \times 2 \times 30 \text{ ps} = 1920 \text{ ps} \approx 1.9 \text{ ns}$$

也就是说，仅这一个加法器就要求系统的节拍不能快于约 1.9 ns 一拍（对应约 0.5 GHz）。现代处理器动辄 3 GHz 以上，每拍只有约 0.3 ns，行波结构显然不堪重任。位宽越大差距越悬殊，这就是必须寻找更快加法器的原因。

4.4 超前进位：用面积买时间

进位链慢，是因为每一级都在**等**上一级。要快，就得设法不等：能否不依赖邻居，直接从原始输入算出每一位的进位？

观察第 i 位，定义两个只依赖本位输入的信号：**生成信号** $G_i = A_i B_i$ ，表示本位自己就造出进位；**传播信号** $P_i = A_i \oplus B_i$ ，表示本位自己不造进位，但若右边送来进位，会把它接力传下去。于是进位满足递推关系

$$C_{i+1} = G_i + P_i C_i$$

即“本位生成，或者本位传播且右边有进位”。这仍是递推，但妙在可以**代数展开**。把 C_1 的式子代入 C_2 ，再代入 C_3 ：

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0$$

展开后的每个式子都**只含各位的 G 、 P 和最初的 C_0** ，不再引用邻居的进位。所有位的进位可以并行地、在固定的两级逻辑内同时算出，串行的等待被彻底消除。这就是**超前进位**（carry-lookahead）的思想。

天下没有免费的午餐：式子越靠高位越长，或门、与门的输入越多，面积和功耗随之上涨；位宽很大时还需分组分层，此处不再展开。但“用更多的门换更短的关键路径”这笔交易，在高速设计中几乎总是划算的。**面积换时间**是数字设计师反复面对的基本抉择，第 6 章讲流水线时，它还会以另一副面孔出现。

顺带完成算术拼图的最后一块：减法不需要新电路。第 1 章的补码告诉我们 $A - B = A + \overline{B} + 1$ ，其中 $\overline{B}$ 表示把 B 按位取反。给加法器的 B 输入端各配一个异或门，全部受同一个“减法”信号 sub 控制（ $\text{sub} = 1$ 时把 B 的每一位翻转），再把 sub 本身接到最低位进位 C_0 （恰好完成“加一”），同一个加法器便加减通吃。这是第 1 章补码设计的红利兑现。

本质要点

本章两块积木各揭示了一个本质。其一，**计算可以化为查表**：香农展开说明任何函数都能装进 mux 树或查找表，“逻辑”与“数据”是一枚硬币的两面。其二，**组合电路的速度由关键路径决定**：功能正确只是及格线，何时稳定同样是设计目标；用并行展开缩短最长路径（如超前进位），是加速的通用套路。

本章小结

- 组合逻辑的输出只由当前输入决定，没有记忆、没有反馈环。
- 多路选择器按选择端取值放行一路输入（ $Y = \overline{S}D_0 + SD_1$ ），是硬件的 if；依据香农展开 $F = \overline{x}F|_{x=0} + xF|_{x=1}$ ，mux 树与查找表可实现任意函数，FPGA 由此而生。
- 半加器完成一位加法（ $S = A \oplus B$, $C = AB$ ）；全加器多收一个进位输入，其本位和是三输入异或，进位输出恰是多数表决。
- 行波进位加法器逐位传递进位，关键路径为整条进位链，延迟随位宽线性增长。
- 超前进位用生成信号 $G_i = A_i B_i$ 与传播信号 $P_i = A_i \oplus B_i$ 把递推式代数展开，各位进位并行算出，以面积换时间。补码让同一个加法器加减通吃。

思考题（答案见附录 A）

- 4.1 只用二选一多路选择器（数据端允许接常数 0、1 或信号），分别实现非门、与门、或门，并由此说明二选一 mux 是功能完备的。
- 4.2 用代数方法证明全加器进位的两种表达式等价： $AB + C_{in}(A \oplus B) = AB + AC_{in} + BC_{in}$ 。提示：把 $A \oplus B = A\overline{B} + \overline{A}B$ 代入展开，再设法补出缺少的部分。
- 4.3 64 位行波进位加法器，每级进位延迟为 2 个门延迟，每个门延迟 25 ps。求进位链总延迟，并估算它允许的最高时钟频率。若换成理想化的超前进位结构（假设从输入到所有进位稳定共 6 个门延迟），速度提高到约多少倍？
- 4.4 在“加法器加异或阵列”的加减两用电路中，控制信号 sub 同时接到每个异或门和最低位进位 C_0 。请解释：为什么 $\text{sub} = 1$ 时电路恰好算出 $A - B$ ？ $\text{sub} = 0$ 时它算什么？

第五章

记忆的诞生：锁存器与触发器

到目前为止，我们的电路都“活在当下”：输入一撤，输出即逝，昨天发生过什么无从知晓。可是几乎一切有意义的计算都离不开记忆：累加要记住部分和，通信要记住走到协议的哪一步。本章从一个反馈环出发，见证记忆在电路中诞生，然后一步步驯服它：先解决“怎么存”，再解决“何时存”，最后直面数字世界最幽微的角落：亚稳态。

5.1 反馈：记忆的种子

组合逻辑为什么没有记忆？因为信号一路向前流，流过就不回头。让电路“记住”的唯一办法，是把输出绕回输入，让信号咬住自己的尾巴。

最简单的尝试：两个反相器首尾相连成环，如图 5.1 左。设法分析它：若节点 Q 为 1，经右边反相器得 $\overline{Q} = 0$ ，再经左边反相器回到 $Q = 1$ ，自洽！若 Q 为 0，同样自洽。这个环有两个彼此稳定的状态，停在哪个就一直是哪个：一个环，记住了一个比特。这种“非此即彼、驻留其一”的性质称为双稳态 (bistability)，它是一切静态存储的原型：芯片内部的 SRAM 存储单元，核心正是这样一对交叉咬合的反相器。

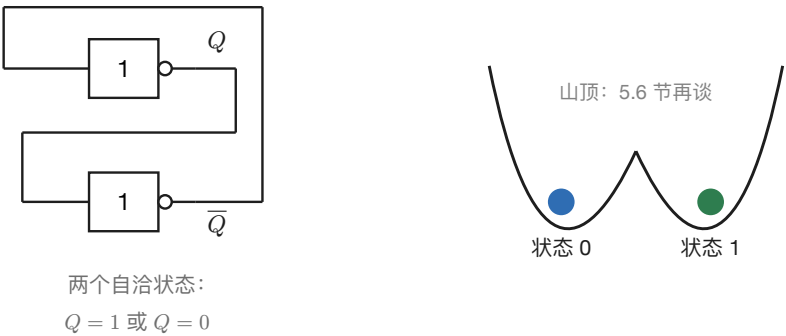
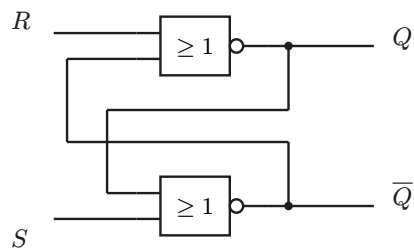


图 5.1 双稳态。左：两个反相器咬尾成环，停在哪个状态就保持哪个。右：像一颗球落在双谷地形中，两个谷底都安稳。

但这个环有个致命缺陷：没有门可进。它固执地保持着出生时的状态，我们却没有任何手段改写它。接下来的所有工作，都是在给这个环开门。

5.2 SR 锁存器：可以改写的记忆

把环里的两个反相器各换成一个或非门，多出来的输入就是“门”。这个电路叫 SR 锁存器 (SR latch)，如图 5.2：S 是置位端 (set，写 1)，R 是复位端 (reset，写 0)。



交叉反馈：每个门的输出咬住对方的输入

图 5.2 用或非门构成的 SR 锁存器。S、R 都为 0 时反馈环保持原状；拉高其中一个即可改写。

分析它只需要记住或非门的脾气：**任一输入为 1，输出立刻变 0；两个输入都为 0 时，输出是另一输入的反。**

- $S = R = 0$: 两个门都听凭反馈环自己转，电路**保持**原有状态。这正是双稳态环的本色。
- $S = 1, R = 0$: 下门输出被压成 0（即 $\overline{Q} = 0$ ），上门两输入全为 0，于是 $Q = 1$ 。**写入了 1**。撤掉 S 后状态自持。
- $S = 0, R = 1$: 对称地，Q 被压成 0，**写入了 0**。
- $S = R = 1$: 两个输出同时被压成 0，既非“记 1”也非“记 0”，而且两端同时撤销时电路何去何从取决于两个门谁快谁慢，结果不可预测。这一输入组合被宣布为**禁用**。

S	R	行为
0	0	保持现状（记忆）
1	0	置 1
0	1	置 0
1	1	禁用

记忆可以改写了，但还有两处不如意：存在禁用组合，而且 S、R 随时都能改写状态，电路毫无“防守”。下一步同时解决这两个问题。

5.3 D 锁存器：数据与时机分离

换一个视角提需求：我们想要的其实是“把**数据 D**在**允许的时机**存进去”。于是给 SR 锁存器加一层门卫：数据 D 与使能信号 EN 通过两个与门生成 $S = EN \cdot D$ 、 $R = EN \cdot \overline{D}$ 。这个电路称为 **D 锁存器**（D latch）。

它的行为可以概括成一句话：**EN = 1 时，输出跟着 D 走（透明）；EN = 0 时，锁住最后一刻的值（保持）**。注意 S 与 R 现在永远不会同时为 1，禁用组合被顺手根除了。图 5.3 是它的波形。

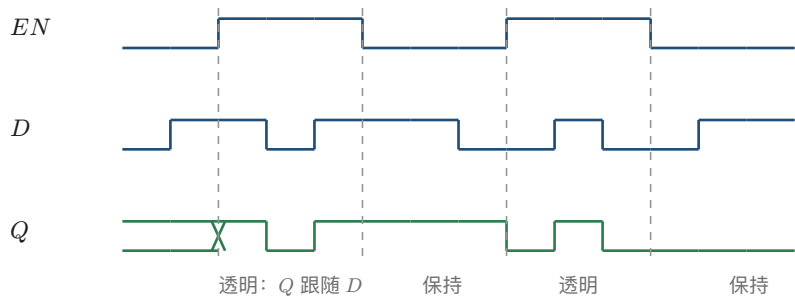


图 5.3 D 锁存器的波形。EN 为高的窗口内输出跟随输入（透明），窗口关闭瞬间锁住当时的值。

看起来大功告成？把它放进真实系统立刻露馅。设想用一串锁存器搭一条流水线，所有 EN 接同一个控制信号：EN = 1 的窗口期间，**所有锁存器同时透明**，数据会像穿过一排敞开的闸门一样，一口气冲过好几级，完全不听指挥。透明是锁存器的天性，也是它的原罪：**窗口再窄也是一段时间，而一段时间里能发生的事太多了**。

我们真正需要的，是把“一段时间”压缩成“一个瞬间”：让存储元件只在某个**时刻**看一眼输入，其余时间一律充耳不闻。

5.4 边沿触发：主从结构与 D 触发器

诀窍是把两个 D 锁存器串起来，让它们的使能**互补**：前一级（主）在时钟 CLK = 0 时透明，后一级（从）在 CLK = 1 时透明，如图 5.4。

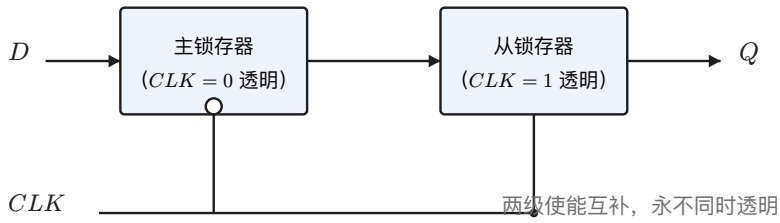


图 5.4 主从 D 触发器。时钟低电平时主级采样、从级紧闭；时钟跃升为高的一瞬，接力棒交给从级。

推演一遍。CLK = 0 期间：主级透明，安静地跟随 D；从级紧闭，输出纹丝不动。时钟跳变为 1 的**一瞬间**：主级关门，把跳变前最后一刻的 D 冻结；从级同时开门，把这个冻结值放行到输出。CLK = 1 的其余时间：主级紧闭，D 再怎么变都进不来。两级永不同时透明，数据每个时钟周期恰好前进一级，绝无一冲到底。

从外面看，这个组合器件的行为极其简洁：**只在时钟由 0 变 1 的上升沿那一瞬采样 D，并保持到下一个上升沿**。这就是**边沿触发**，这个器件就是大名鼎鼎的**D 触发器**（D flip-flop），数字芯片中数量最多的存储元件。它的符号是一个方框：左侧 D 输入、

右侧 Q 输出，左下角的小三角形专门标记“边沿敏感”的时钟端。图 5.5 直观对比了锁存器与触发器面对同一串输入的表现。

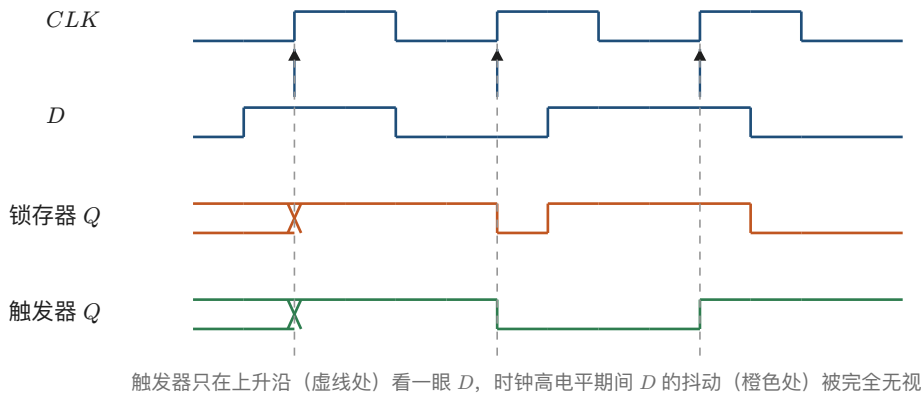


图 5.5 同一输入下锁存器与触发器的对比。锁存器在整个高电平窗口跟随 D ，触发器只认上升沿那一瞬。

定义 时序逻辑与寄存器

含有存储元件、输出取决于输入历史的电路称为时序逻辑 (sequential logic)。若干个 D 触发器共用一个时钟并排工作，合称一个寄存器 (register)，一次可存入一个多位的值。“32 位处理器”里的通用寄存器，就是 32 个 D 触发器的并排。

5.5 时钟：把连续时间切成节拍

D 触发器把“何时改变状态”收拢到了一个信号的上升沿。顺理成章的下一步，是让全芯片所有触发器共用同一个时钟。这个看似朴素的决定，是数字设计中影响最深远公约，称为同步设计范式。

它买到的东西超乎想象。两次上升沿之间，所有触发器都端坐不动，组合逻辑在它们之间自由演算：信号翻腾、竞速、抖动（这类瞬态毛刺称为冒险，hazard），都无所谓，只要在下一个上升沿到来之前一切尘埃落定，触发器采到的就是正确答案。时钟像考场的收卷铃：铃响之间你怎么打草稿、涂改多少次都没人管，铃响那一刻卷面正确即可。混乱被圈进了节拍内部，节拍之间的世界秩序井然。

本质要点

时钟的本质是把连续的时间离散化，正如数字抽象把连续的电压离散化。第 1 章我们在电压轴上划区间，扔掉了幅度细节，换来抗噪；本章我们在时间轴上划节拍，扔掉了节拍内的过程细节，换来对瞬态混乱的免疫。两次离散化，同一个哲学：主动放弃无关紧要的精确，换取绝对可靠的判断。数字系统的全部可靠性，就建立在这两次“放弃”之上。

当然，“在下一个沿到来之前尘埃落定”是一句需要兑现的承诺，兑现的规则（建立时间、保持时间）正是第 6 章的主题。在那之前，还欠一个坦白：有一种失效，连触发器也无法幸免。

5.6 亚稳态：山顶上的球

回看图 5.1 右边的双谷地形：两个谷底对应两个稳定状态，而两谷之间还有一个**山顶**。一颗球恰好停在山顶上，理论上可以永远保持平衡，实际上任何风吹草动都会让它倒向某一边，但**何时倒、倒向哪边，无法预知**。

双稳态电路真的存在这个山顶：反馈环里两个节点的电压恰好停在高低之间的平衡点上。若在触发器采样的瞬间，输入 D 正巧在翻转（采样窗口被侵犯，见第 6 章），触发器就可能被“推到山顶”：输出在合法的 0 与 1 之间徘徊，或经过异常漫长的时间才滚落谷底。这种状态称为**亚稳态**（metastability）。它危险之处在于：下游若在球落定之前读取，不同的门可能读出不同的值，整个系统的逻辑一致性瞬间崩塌。

什么时候会撞上它？在一个时钟域内部，第 6 章的时序纪律可以保证采样窗口永不被侵犯。真正躲不开的是**异步输入**：按钮按下的时刻、另一块芯片发来的信号、不同时钟域之间的数据，它们与本地时钟毫无约定，总有一天恰好踩中采样瞬间。**亚稳态不可根除**，这是双稳态物理注定的；工程能做的，是让它的危害小到宇宙尺度上都难得一见。

数学加油站 指数衰减：为什么“多等一拍”如此有效

处在亚稳态的电路会自发地向某个稳定态滚落，其“仍未落定”的概率随等待时间**指数**下降：

$$P(\text{等待 } t \text{ 后仍未落定}) \approx e^{-t/\tau}$$

其中 τ 是由电路增益与电容决定的时间常数，先进工艺中约为几皮秒到几十皮秒。指数函数衰减的迅猛程度远超直觉：设 $\tau = 30 \text{ ps}$ ，多等 1 ns ，未落定概率就缩小 $e^{1000/30} \approx e^{33.3} \approx 3 \times 10^{14}$ 倍。也就是说，**多给一个时钟周期的冷静时间，故障率可以从“天天发生”压到“亿万年一遇”**。工程上用平均无故障时间（MTBF）来量化这件事，它随等待时间指数增长。

据此得到标准解法：**同步器**（synchronizer）。异步信号不许直接使用，必须先串过两级触发器，如图 5.6。第一级明知可能进入亚稳态，没关系；到第二级采样时，差不多整整一个时钟周期已经过去，第一级“仍未落定”的概率已被指数压到忽略不计。第二级的输出，才允许进入芯片内部。要求更苛刻的场合再加第三级，MTBF 又乘上一个天文数字。

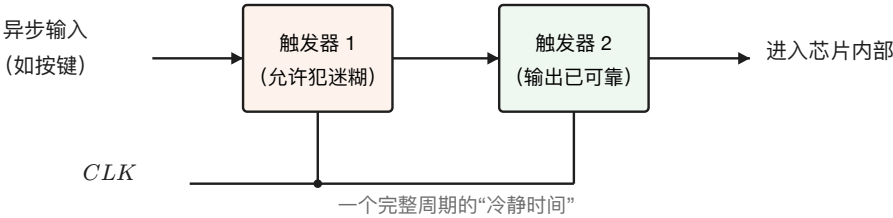


图 5.6 两级同步器。第一级可能亚稳，第二级采样前它有近一个周期的时间落定，风险被指数压低。

本章小结

- 记忆源于反馈：两个反相器咬尾成环即双稳态，能驻留一个比特，但无法改写。
- SR 锁存器给环开了写入口（置 1、置 0、保持），代价是存在禁用组合；D 锁存器分离数据与时机并根除禁用组合，但“电平透明”会让数据一冲到底。
- 主从结构把两段互补的透明期压成一个瞬间，得到边沿触发的 D 触发器：只在时钟上升沿采样，其余时间充耳不闻。
- 同步设计范式让全芯片共用时钟节拍：节拍之间组合逻辑尽情演算，毛刺与竞速只要在下个沿之前尘埃落定即无害。时钟对时间的离散化与数字抽象对电压的离散化一脉相承。
- 采样瞬间恰逢输入翻转可能引发亚稳态（山顶的球），异步输入使其不可根除；两级同步器利用“未落定概率随时间指数衰减”把风险压到宇宙尺度罕见。

思考题（答案见附录 A）

- 5.1 两个反相器成环有两个稳定状态。若改成三个反相器成环，请分析：还存在稳定状态吗？这个电路会表现出什么行为？它有什么实际用途？
- 5.2 SR 锁存器中令 $S = R = 1$ ，两个输出都变为 0；随后两个输入恰好同时撤回 0。请解释此时电路为何陷入类似“山顶的球”的处境，这与 5.6 节的亚稳态有什么联系？
- 5.3 有人图省事，用 D 锁存器（而非触发器）串成移位链，所有使能端接同一时钟。请描述使能为高的窗口内会发生什么，并说明为什么窗口改得再窄也不能从根本上解决问题。
- 5.4 同步器为什么普遍采用两级而不是一级？从 MTBF 的指数公式出发，说明再加第三级的收益与代价。若时钟频率提高一倍（周期减半），同步器的可靠性会往哪个方向变化，为什么？

第六章

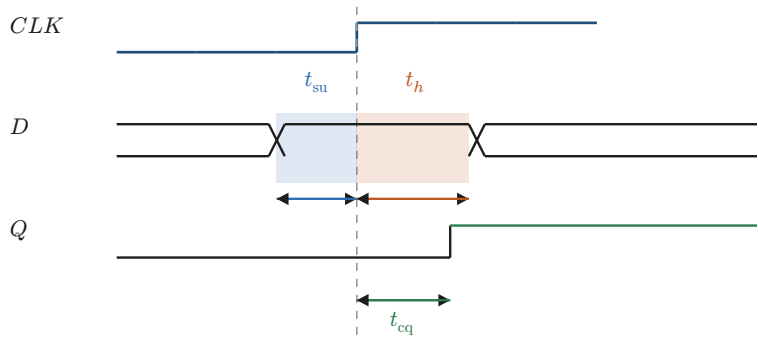
时间的纪律：建立、保持与流水线

上一章许下了一个诺言：“只要在下一个时钟沿之前尘埃落定”。本章把这句话变成可以计算的数学。你将理解芯片规格表上“3 GHz”这个数字从哪里来，为什么它不能随意调高，以及工程师最常用的提速魔法：流水线。这一章的公式极少，却是数字工程师每天赖以谋生的那几条。

6.1 触发器的契约：三个时间参数

第5章把D触发器描述成“在上升沿一瞬采样”。物理的真相略微粗糙一些：采样不是无限短的一瞬，触发器内部的晶体管需要一小段时间才能把输入值可靠地咬住。于是，每个真实的触发器都向使用者开出一份契约，包含三个参数（见图 6.1）：

- **建立时间** t_{su} (setup time)：数据必须在时钟沿之前 t_{su} 就位并保持不动；
- **保持时间** t_h (hold time)：数据在时钟沿之后还要继续纹丝不动 t_h ；
- **时钟到输出延迟** t_{cq} (clock-to-Q delay)：时钟沿过后，新值要再过 t_{cq} 才出现在 Q 端。



着色区间内 D 必须稳定；新值在 t_{cq} 之后才到达 Q

图 6.1 触发器的时序契约。时钟沿前 t_{su} 、沿后 t_h 构成采样窗口，窗口内数据必须稳定。

把 t_{su} 与 t_h 夹住的区间想象成拍照的快门：按下快门前摆好姿势（建立），快门未合之前别乱动（保持），照片洗出来还要等一会儿（ t_{cq} ）。违反前两条，照片就是糊的，那正是第5章所说的亚稳态风险。

6.2 建立约束：主频的由来

现在把契约用在真实电路上。同步系统中最典型的一段路，如图 6.2：数据从触发器 FF1 出发，穿过一团组合逻辑，赶往下一个触发器 FF2。两个触发器共用时钟，节拍周期为 T_{clk} 。

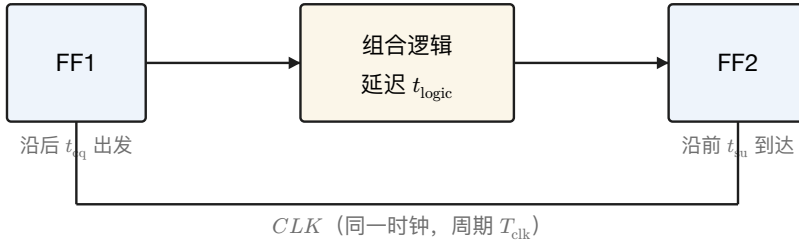


图 6.2 同步系统的基本路径。一个节拍内，数据要完成“出发、穿越、就位”三件事。

在第 n 个上升沿，FF1 采入新值；经过 t_{cq} ，新值出现在 FF1 输出；再经过组合逻辑最长 t_{logic} 的跋涉，抵达 FF2 的输入；而它必须赶在第 $n+1$ 个上升沿之前 t_{su} 就位。三段行程加起来不能超过一个周期，这就是**建立约束**（setup constraint）：

$$T_{\text{clk}} \geq t_{\text{cq}} + t_{\text{logic}}^{\text{max}} + t_{\text{su}}$$

移项即得最高频率 $f_{\text{max}} = 1/T_{\text{min}}$ 。注意式中取的是**最慢**的组合路径：全芯片千千万万条路径中最慢的那条（关键路径，第 4 章已见过它）决定了整块芯片的节拍上限。木桶原理，一条短板封顶全局。

例 6.1 计算最高主频

某设计中触发器 $t_{\text{cq}} = 50 \text{ ps}$ ， $t_{\text{su}} = 40 \text{ ps}$ ，最长组合路径延迟 600 ps 。求最高时钟频率。
解

$$T_{\text{min}} = 50 + 600 + 40 = 690 \text{ ps}, \quad f_{\text{max}} = \frac{1}{690 \times 10^{-12} \text{ s}} \approx 1.45 \text{ GHz}$$

若想冲上 2 GHz ($T = 500 \text{ ps}$)，组合逻辑最多只许 $500 - 90 = 410 \text{ ps}$ ：要么优化关键路径上的逻辑（如换用超前进位），要么，把逻辑砍成两段，这就是本章稍后的流水线。这个算式也解释了“超频”：厂商标称 1.45 GHz 留有余量（工艺、电压、温度的波动都要预算在内），用户强强调高频率，本质是吃掉安全余量；一旦某条路径在某个温度下迟到，就会采到错误数据。

6.3 保持约束：另一半契约

建立约束管“别迟到”，契约里还有一条“别抢跑”。同一个时钟沿，FF1 采入新值后，新值会在 t_{cq} 后冲出闸门、穿过组合逻辑冲向 FF2；如果这条路太**短**，新值可能在 FF2

还没保住**上一拍**数据时（即沿后 t_h 之内）就赶到，把旧数据冲掉。要求新值到达不早于 t_h ，得到**保持约束**（hold constraint）：

$$t_{cq} + t_{logic}^{min} \geq t_h$$

请仔细品味它与建立约束的三点不同：它盯的是**最短路径**（ t_{logic}^{min} ）；它描述的是**同一个**时钟沿前后的赛跑，而不是相邻两个沿；因此**它与时钟周期完全无关**。

常见误区

“时序不满足？降频试试。”这句急救咒语只对建立违例有效：周期拉长，迟到自然消失。**保持违例降频毫无用处**，因为约束里根本没有 T_{clk} 。一块保持时间违例的芯片，无论跑多慢都错，等于报废。这也是为什么设计流程对保持违例如临大敌：常规修法是在过短的路径上刻意串入缓冲器“添堵”，把太快的信号拖慢到安全线之后。

顺带补一个真实世界的注脚。同一个时钟沿“同时”到达所有触发器只是理想；实际上时钟经过不同长度的走线到达各触发器的时刻略有先后，这个偏差称为**时钟偏斜**（clock skew）。偏斜会侵蚀两条约束的余量（方向恰好相反：接收端时钟偏早伤建立、偏晚伤保持），所以芯片里要专门构建平衡的“时钟树”把偏斜压到最小，这是第 9 章物理设计的重要工序。

6.4 流水线：用并行换节拍

回到提速问题。例 6.1 说关键路径 600 ps 封顶 1.45 GHz，若这段逻辑实在无法再优化，还有一招：**在半路插一排触发器，把长路截成两段短路**。这就是**流水线**（pipeline），数字设计中威力最大、使用最广的架构手法，如图 6.3。

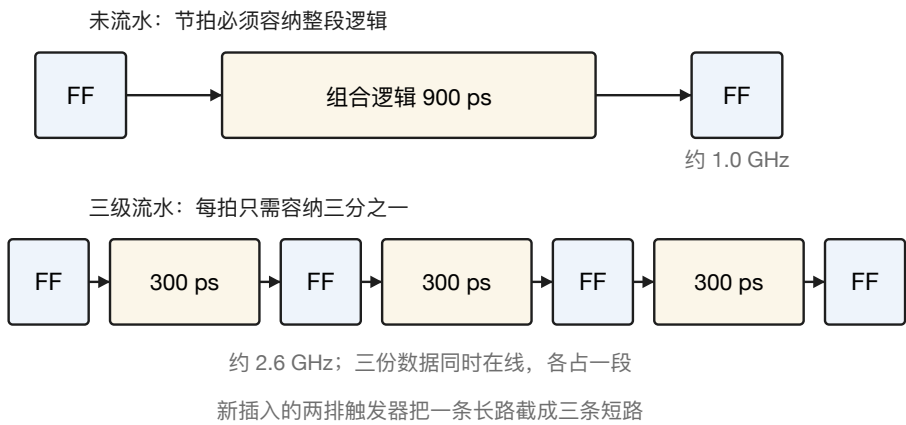


图 6.3 流水线改造。截断关键路径换来高节拍；代价是寄存器面积与更长的单笔延迟。

它的精髓不在“把活变少”，而在“让人手不闲”。想象洗衣店的洗、烘、叠三道工序，各需 20 分钟。一件订单从头做到尾要 60 分钟，这是**延迟**（latency）。但若三道工序各配一人流水作业：第一件在烘的同时，第二件已经在洗。稳定后**每 20 分钟就有一件完**

工，这是吞吐率（throughput）。电路同理：插入两排流水线寄存器后，三段逻辑里同时躺着三笔不同的数据，各自演算各自的一段，互不干扰。

例 6.2 流水线的账本

接例 6.1 的思路：某路径组合逻辑共 900 ps，触发器开销 $t_{cq} + t_{su} = 90$ ps。比较不分级与均匀三级流水。

解 不分级： $T = 900 + 90 = 990$ ps, $f \approx 1.0$ GHz；每秒完成约 10 亿笔。

三级流水：每级 $T = 300 + 90 = 390$ ps, $f \approx 2.56$ GHz；每秒完成约 25.6 亿笔，吞吐率约 2.5 倍。

再看代价。单笔数据的延迟：从 990 ps 变为 $3 \times 390 = 1170$ ps，反而变慢了（多付了两次触发器开销）；外加两排寄存器的面积与功耗。流水线是一笔“牺牲个体、成全总量”的交易，几乎所有现代处理器都把指令执行切成十几级甚至更深的流水。

还应注意收益的天花板：级数越深，每级里 90 ps 的固定开销占比越大（390 ps 里占 23%），继续切分的边际收益递减；且各级必须切得均匀，最慢的一级仍然封顶全局。

本质要点

同步设计的时间账本浓缩为两行：**建立约束** $T_{clk} \geq t_{cq} + t_{logic}^{max} + t_{su}$ ，盯最慢路径，决定主频；**保持约束** $t_{cq} + t_{logic}^{min} \geq t_h$ ，盯最快路径，与主频无关。一切时序优化，本质都是在改写这两个不等式里的某一项：超前进位缩小 t_{logic}^{max} ，流水线把一个大 t_{logic} 拆成几个小的，修保持则是垫高 t_{logic}^{min} 。

本章小结

- 触发器的契约有三个参数：数据须在沿前 t_{su} 就位、沿后 t_h 不动，新值沿后 t_{cq} 才输出；侵犯采样窗口就有亚稳态风险。
- 建立约束 $T_{clk} \geq t_{cq} + t_{logic}^{max} + t_{su}$ ：最慢路径决定最高主频（木桶原理）。超频吃的是安全余量。
- 保持约束 $t_{cq} + t_{logic}^{min} \geq t_h$ ：同一时钟沿前后的赛跑，与周期无关，降频无解；修法是给过短路径添延迟。时钟偏斜会同时侵蚀两条约束的余量。
- 流水线在长路径中插入寄存器：吞吐率成倍提升，单笔延迟与面积略增；级数受触发器固定开销与分段均匀性制约，收益递减。

思考题（答案见附录 A）

6.1 某设计 $t_{cq} = 60$ ps, $t_{su} = 50$ ps, $t_h = 30$ ps，最长组合路径 800 ps，最短组合路径 20 ps。求最高频率，并检查保持约束是否满足；若不满足，说出一种修复手段。

6.2 解释为什么工程师说“建立违例是性能问题，保持违例是功能问题”。两者分别用什么手段修复？

6.3 把一段 1200 ps 的组合逻辑改造成 n 级均匀流水线，触发器开销 $t_{cq} + t_{su} = 80$ ps。写出频率关于 n 的表达式，算出 $n = 1, 2, 4, 8$ 时的频率，并观察收益递减；说说除了固定开销，现实中还有什么因素阻止“无限加深流水线”。

6.4 时钟偏斜使 FF2 的时钟比 FF1 晚到 δ 。请分别改写建立与保持两条约束（把 δ 加进去），并据此说明：为什么说“偏斜是把双刃剑，帮了建立就一定伤保持”。

第七章

有限状态机：让电路按剧本行事

电路已经会记忆、会打节拍了，但它还只是“零件”。要让电路完成一件有头有尾的任务，比如识别一段暗号、控制一部电梯、执行一条协议，就需要给它写剧本：什么情境下做什么、然后进入什么情境。这份剧本的正式名字叫有限状态机。它是数字系统的“大脑皮层”，处理器的控制器、通信芯片的协议引擎，本质上都是状态机。本章用一个完整的例子，把状态机从一句需求做到一张电路图。

7.1 状态机的骨架

先给出概念。**有限状态机**（Finite State Machine, FSM）由三样东西组成：

- 一组有限的**状态** (state)：系统可能处的“情境”，即它需要记住的全部历史的摘要；
- **转移规则**：在每种状态下收到每种输入，下一拍去哪个状态；
- **输出规则**：什么时候对外界说什么。

落到电路上，骨架惊人地简单，如图 7.1：一个**状态寄存器**（一组 D 触发器）保存当前状态；一团组合逻辑（**次态逻辑**）根据当前状态和输入算出下一个状态，等待时钟上升沿存入；另一团组合逻辑（**输出逻辑**）产生输出。第 5 章的触发器和第 3、4 章的组合逻辑，在这里合体了。

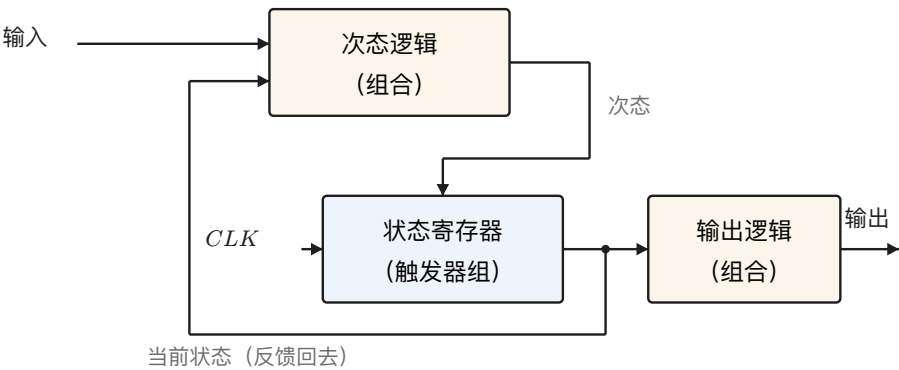


图 7.1 状态机的通用骨架：寄存器存“现在”，次态逻辑算“未来”，时钟沿把未来变成现在。

每个时钟沿，机器恰好走一步。状态机的设计，归根结底是回答两个问题：**需要哪些状态**，以及**状态之间怎么走**。

输出规则有两个流派：输出只由当前状态决定的，称为**摩尔型**（Moore）状态机；输出由当前状态和当前输入共同决定的，称为**米利型**（Mealy）状态机。摩尔型输出与

时钟对齐、干净稳重；米利型反应快一拍、但输出会跟着输入抖动。本章用更常见的摩尔型讲解。

7.2 完整实战：序列检测器

需求：设计一个电路，监听一条每拍送来一位的串行数据线 X ，每当最近三位恰好是“101”时，输出 Z 拉高一拍。允许重叠，例如输入“10101”应报告两次（第 3 拍和第 5 拍）。这类“暗号识别”电路在通信协议中随处可见（识别帧头、同步字）。

第一步：设计状态。状态是“历史的摘要”，问题是：为了判断暗号，需要记住多少历史？不必记住全部输入，只需记住到目前为止，已经匹配了暗号的多长前缀。于是状态呼之欲出：

- S_0 ：一无所获（最近的输入对匹配毫无帮助）；
- S_1 ：已匹配前缀“1”；
- S_2 ：已匹配前缀“10”；
- S_3 ：刚刚凑齐“101”，本拍输出 $Z = 1$ 。

第二步：画状态图。逐个状态问“来 0 去哪、来 1 去哪”，答案汇成图 7.2。这里最容易错的是“失配”走向，要点是：失配不一定回到零，新来的几位可能已经是暗号的新开头。例如在 S_3 （刚匹配“101”）再来一个 1，最近两位是“11”吗？不，是“01”的后一半加这个 1，即最近一位“1”可作新前缀，去 S_1 ；再来 0，最近两位“10”恰是前缀，去 S_2 。

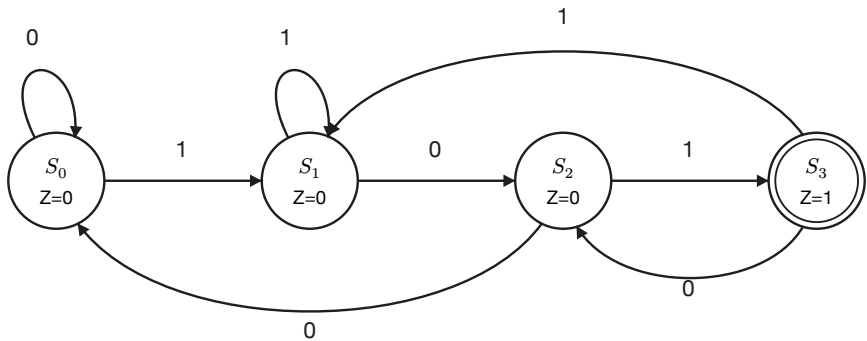


图 7.2 “101” 序列检测器的状态图（摩尔型）。圈内标注状态名与该状态的输出；箭头上标注触发转移的输入。

第三步：列状态表。把图翻译成表，机械而务必细心：

当前状态	含义	$X = 0$ 时次态	$X = 1$ 时次态
S_0	一无所获	S_0	S_1
S_1	已匹配“1”	S_2	S_1
S_2	已匹配“10”	S_0	S_3
S_3	已匹配“101”，输出 1	S_2	S_1

第四步：给状态编码。四个状态用两个触发器（记为 Q_1Q_0 ）即可编码： $S_0 = 00$ ， $S_1 = 01$ ， $S_2 = 10$ ， $S_3 = 11$ 。

第五步：求次态与输出逻辑。把状态表按编码改写成真值表（输入为 Q_1 、 Q_0 、 X ，输出为次态 D_1 、 D_0 ，它们接到两个触发器的 D 端），然后用第 3 章的卡诺图化简。此处直接给出结果，推导留作练习（思考题 7.1）：

$$D_1 = Q_0\overline{X} + Q_1\overline{Q_0}X, \quad D_0 = X, \quad Z = Q_1Q_0$$

$D_0 = X$ 尤其漂亮：观察编码会发现，四个状态中编码低位为 1 的（ S_1 、 S_3 ）恰好都是“刚收到 1”的状态，所以下一拍的低位就是本拍的输入。好的编码常常带来意外的化简。

第六步：落成电路。两个 D 触发器加上按上式连线的几个门，就是全部电路，结构正是图 7.1 的骨架。至此，一句自然语言的需求，走完了状态、表格、代数、电路的全程。

最后务必**推演验证**。用输入串“1 0 1 0 1”走一遍（设初始在 S_0 ）：

拍	本拍输入 X	拍前状态	拍后状态	拍后输出 Z	说明
1	1	S_0	S_1	0	收到“1”
2	0	S_1	S_2	0	收到“10”
3	1	S_2	S_3	1	凑齐“101”，报告
4	0	S_3	S_2	0	重叠：末尾“10”仍是前缀
5	1	S_2	S_3	1	再次凑齐“101”

输入“10101”报告两次，重叠检测正确，与需求吻合。

7.3 状态编码的门道

第四步“怎么给状态编号”看似随意，实则是一个真正的设计决策。两种主流方案各有拥趸：

- **二进制编码**： n 个状态用 $\lceil \log_2 n \rceil$ 个触发器，寄存器最省。缺点是次态逻辑往往较复杂（多个状态位纠缠在同一个表达式里）。
- **独热编码**（one-hot）：每个状态独占一个触发器， n 个状态用 n 个触发器，任何时刻恰有一位为 1。触发器多了，但“当前是否处于状态 S_k ”直接就是一根线，次态与输出逻辑常常简单得多、也更快。

在触发器充裕的 FPGA 上，独热编码是常见默认；在寸土寸金的定制芯片上，二进制或更精心的编码更受青睐。这仍是那笔熟悉的交易：**面积与速度、寄存器与组合逻辑之间的权衡**，第 4 章与第 6 章的主题在此再次现身。

另有一件事关生死的细节：**复位**。上电瞬间触发器的初值是随机的，状态机可能生在任何编码上，甚至是没有定义的编码（例如独热编码中“两个 1”的非法组合）。所以每个状态机都必须有复位信号，把它强制送入指定的初始状态。忘记复位的状态机，是新手电路“上电即疯”的头号原因。

本质要点

状态机设计的灵魂在第一步：**状态是对历史的最小充分摘要**。摘要太粗，任务无法完成；太细，状态数爆炸。“已匹配前缀的长度”这类摘要之所以精妙，是因为它恰好扔掉了与未来无关的一切历史。这个“压缩历史”的视角，与第 1 章“扔掉电压细节”、第 5 章“扔掉节拍内过程”一脉相承：**设计的艺术，是决定什么可以忘记**。

本章小结

- 状态机 = 状态寄存器 + 次态逻辑 + 输出逻辑；每个时钟沿走一步。摩尔型输出只看状态，米利型还看输入。
- 设计流程：定状态（历史的最小摘要）→ 画状态图 → 列状态表 → 编码 → 化简求 D 、 Z 表达式 → 落成电路 → 推演验证。
- 失配转移不一定回初态：新输入可能已构成模式的新前缀（重叠检测）。
- 二进制编码省触发器，独热编码简化逻辑、速度快；选择是面积与速度的权衡。状态机必须复位到确定初态。

思考题（答案见附录 A）

- 7.1 按第五步的编码把状态表展开成真值表（三个输入 Q_1 、 Q_0 、 X ，输出 D_1 、 D_0 、 Z ），用卡诺图化简，验证正文给出的三个表达式。
- 7.2 把需求改为**不允许重叠**（报告一次后从头再来，“10101”只报告一次）。状态图哪几条边需要修改？画出修改后的状态表。
- 7.3 设计“110”检测器（允许重叠）的状态图：写出各状态含义与全部转移。它与“101”检测器的状态数相同吗？
- 7.4 一个 12 个状态的控制器，二进制编码与独热编码各需要几个触发器？若采用独热编码，如何用 一个与门实现“当前处于 S_5 且输入 r 为 1”这个条件？这说明独热编码的什么优势？

第八章

用代码描述硬件：Verilog 与 RTL 思维

手工画门到此为止。真实的芯片动辄百万行逻辑，靠的不是原理图，而是硬件描述语言：工程师用代码描述电路，综合工具把代码翻译成门。本章学习最流行的硬件描述语言 Verilog。请先在心里默念三遍本章章训：写的不是程序，是电路。语法长得像 C，骨子里完全是另一种生物；理解这一点，就理解了本章的一切，包括那个折磨过无数初学者的问题：阻塞与非阻塞赋值。

8.1 描述，而不是指令

普通程序是**指令的序列**：一条接一条执行，同一时刻只做一件事。而电路是**结构**：所有的门时时刻刻都在工作，天生并行。第 3 章的多数表决器里，三个与门可没有先后之分。

因此，**硬件描述语言**（Hardware Description Language, HDL）写下的每一行，不是“做什么”，而是“存在什么”：这里有一团逻辑，它的输出恒等于某个表达式。代码的先后顺序大多无关紧要，就像原理图上元件摆放的位置不改变电路。读 Verilog 时，脑中浮现的应该是门和线，而不是流程图。

写好的代码有两个去处。**仿真器**把它当作可执行的模型，在计算机上推演波形，供你验证对错（8.6 节）；**综合器**（synthesizer）把它翻译成真实的门网表，送往制造流程（第 9 章）。两者共同的语义基础，是一种介于代数与门之间的抽象层级：

定义 寄存器传输级（RTL）

用“寄存器 + 寄存器之间的组合逻辑”来描述电路的抽象层级，称为寄存器传输级（Register-Transfer Level, RTL）。一份 RTL 代码回答一个问题：**每个时钟沿，每个寄存器该装入什么函数的结果**。第 5、6、7 章反复出现的“触发器、组合云、触发器”图景（图 8.1），正是 RTL 的世界观。

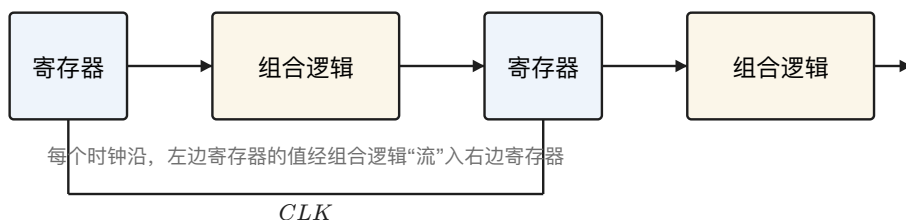


图 8.1 RTL 的世界观：数据在寄存器之间逐拍传输，组合逻辑负责途中的加工。

8.2 模块与组合逻辑

Verilog 的基本单位是**模块** (module)：一个有名字、有端口的电路块。先看熟人，第 3 章的多数表决器：

```

module majority (           // 模块名
    input wire a, b, c, // 三个一位输入
    output wire f        // 一位输出
);
    assign f = (a & b) | (a & c) | (b & c);
endmodule
  
```

`assign` (连续赋值) 是组合逻辑最直白的写法：`f 永远等于`右边表达式，右边任何信号一变，`f` 随之而变，如同真的导线和门。`&`、`|`、`~`、`^` 分别对应与、或、非、异或。`wire` 意为“线网”，就是一根导线。

分支较多的组合逻辑，用 `always` 块配合 `if`、`case` 更顺手。写第 4 章的二选一 `mux`：

```

module mux2 (
    input wire d0, d1, s,
    output reg y           // 在 always 块中被赋值的信号要声明为 reg
);
    always @(*) begin      // @(*): 右边任何信号变化都重新求值
        if (s) y = d1;
        else y = d0;
    end
endmodule
  
```

两个语法点。其一，`@(*)` 是敏感列表，星号表示“块内用到的所有信号”，组合逻辑一律这么写，手工罗列容易漏。其二，一个重要的历史包袱：

常见误区

reg 不等于寄存器！它只是 Verilog 的规定：“凡在 always 块里被赋值的信号，须声明为 reg 类型”。上面的 mux 里没有任何触发器，y 依然是纯组合逻辑。是否生成触发器，只取决于 always 块由什么触发（电平变化还是时钟沿），与 reg 这个词毫无关系。后继语言 SystemVerilog 已把它改名为 logic，算是官方道歉。

组合 always 块还有一个著名陷阱。若某条路径下忘了给输出赋值，例如只写 if (s) y = d1; 而没有 else，语义就成了“其余情况 y 保持旧值”。保持旧值就是记忆，综合器只好给你造一个锁存器（第 5 章的 D 锁存器），而这多半不是你想要的。规则：组合 always 块里，所有分支都必须给所有输出赋值（或在块首先赋默认值）。

8.3 时序逻辑：让时钟说话

生成触发器的咒语是 posedge（上升沿）。写一个带复位的 8 位计数器：

```
module counter8 (
    input wire      clk,
    input wire      rst_n,    // 低电平有效的复位 (_n 后缀是惯例)
    output reg [7:0] cnt      // [7:0]: 8 位总线, cnt 是 8 个触发器
);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) cnt <= 8'd0;    // 复位: 清零
        else      cnt <= cnt + 8'd1; // 每拍加一, 自然回绕 (模 256)
    end
endmodule
```

always @(posedge clk ...) 的含义：块内的赋值只在时钟上升沿发生，这正是 D 触发器的行为，综合器会为 cnt 生成 8 个触发器和一个加法器。8'd0 表示“8 位宽的十进制 0”。cnt + 1 在 255 之后回绕到 0，正是第 1 章模运算的直接兑现。复位保证上电后从确定值出发（第 7 章的教训）。

注意这里的赋值符号变成了 <=，它就是下一节的主角。

8.4 阻塞与非阻塞：一字之差，两种电路

Verilog 有两种赋值：阻塞赋值 = 与非阻塞赋值 <=。用一个三级移位寄存器（数据每拍向前挪一格）看它们的分歧：

```
// 写法一：非阻塞 <= （正确的移位寄存器）
always @(posedge clk) begin
    q1 <= d;
```



```

q2 <= q1;    // 用的是 q1 的“旧值”
q3 <= q2;
end

```

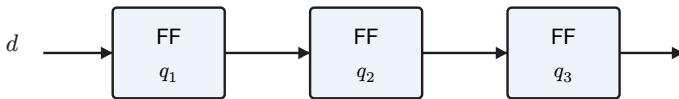
```

// 写法二：阻塞 = （错！三个触发器全被接到 d）
always @(posedge clk) begin
    q1 = d;    // q1 立即变成 d
    q2 = q1;   // 于是 q2 = d
    q3 = q2;   // 于是 q3 = d
end

```

语义区别一句话：**非阻塞先统一采样右边的旧值，再统一更新左边**，三条语句像三个触发器一样同时动作；**阻塞则像 C 语言一样立即生效、顺序执行**，前一条的结果立刻被后一条用上。于是写法一综合出真正的移位链，写法二综合出三个并联的、都装入 d 的触发器，如图 8.2。

非阻塞 <=: 三级移位链



阻塞 =: 三个并联的复制品

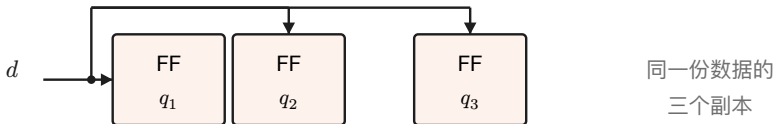


图 8.2 一字之差综合出的两种电路。非阻塞对应“所有触发器同时采样旧值”的物理现实。

为什么非阻塞才是对的？回想第 5 章：真实芯片里，时钟沿到来的一瞬，**所有触发器同时采样各自输入的旧值**，然后同时更新。谁也不比谁先跑。非阻塞赋值正是这一物理现实的语言化身；阻塞赋值的“顺序生效”在硬件里根本没有对应物。由此得到一条几乎不需要例外的军规：

本质要点

时序逻辑（**posedge** 块）一律用非阻塞 **<=**；组合逻辑（**@(*)** 块）一律用阻塞 **=**；同一个块里绝不混用。遵守它，代码的行为与综合出的电路才会一致；违反它，仿真可能侥幸正确而芯片错误，或者相反，这是硬件调试中最阴险的一类不一致。

8.5 状态机进 Verilog

把第 7 章的“101”检测器写成代码，是对前两节最好的总结。业界惯用“三段式”写法，三个部分与图 7.1 的骨架一一对应：

```
module seq101 (
    input wire clk, rst_n,
    input wire x,          // 串行输入
    output wire z           // 检测到 101 时拉高一拍
);

    localparam S0 = 2'b00, S1 = 2'b01,    // 状态编码表
               S2 = 2'b10, S3 = 2'b11;
    reg [1:0] state, next;

    // 一段：状态寄存器（时序，非阻塞）
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) state <= S0;           // 复位进入初态
        else          state <= next;
    end

    // 二段：次态逻辑（组合，阻塞），照抄第 7 章的状态表
    always @(*) begin
        case (state)
            S0: next = x ? S1 : S0;
            S1: next = x ? S1 : S2;
            S2: next = x ? S3 : S0;
            S3: next = x ? S1 : S2;
            default: next = S0;             // 兜底，防综合出锁存器
        endcase
    end

    // 三段：输出逻辑（摩尔型：只看状态）
    assign z = (state == S3);
endmodule
```

请与第 7 章的状态表逐行核对 case 的四行，一一对应。 $x ? S1 : S0$ 是条件运算符，即“ x 为 1 选 S1，否则选 S0”，综合成 mux（第 4 章“硬件的 if”所言非虚）。手工推导的卡诺图化简去哪了？交给综合器了：它做的正是第 3 章的化简，而且比人做得更好。工程师的职责上移到了把状态图写对。

8.6 验证初步：测试平台

代码写完，如何知道对错？答案不是盯着看，而是**仿真**：再写一段代码扮演“考官”，向被测电路灌输入、看输出。这段考官代码称为**测试平台**（testbench）：

```
`timescale 1ns/1ps
module tb;
    reg  clk = 0, rst_n = 0, x = 0;
    wire z;

    // 例化被测电路
    seq101 dut (.clk(clk), .rst_n(rst_n), .x(x), .z(z));

    always #5 clk = ~clk;          // 每 5 ns 翻转：周期 10 ns 的时钟

    initial begin
        #12 rst_n = 1;             // 12 ns 后撤销复位
        @(negedge clk) x = 1;      // 下降沿改输入，避开采样沿
        @(negedge clk) x = 0;
        @(negedge clk) x = 1;      // 至此送入 1 0 1，期待 z=1
        @(negedge clk) x = 0;
        @(negedge clk) x = 1;      // 送完 10101，期待第二次 z=1
        @(negedge clk) x = 0;
        #20 $finish;               // 结束仿真
    end

    // 打印跟踪
    initial $monitor("t=%0t  x=%b  z=%b", $time, x, z);
endmodule
```

运行仿真器（开源的 Icarus Verilog、Verilator 都可以），对照打印或波形窗口检查 z 是否恰好拉高两次。强烈建议读者真的跑一遍，第 7 章的推演表将在波形里活过来。

注意测试平台里出现了 #5（延时 5 纳秒）、initial（只执行一次的过程）这类语句。它们只存在于仿真世界，**不可综合**：芯片里没有“等 5 纳秒”这个门。Verilog 因此分成两个子集：可综合子集描述电路本身，全部语言能力则服务于验证。混淆两者是初学者常见的第二类错误。至于验证为什么在真实项目中比设计本身还费人力，第 9 章见分晓。

本章小结

- HDL 描述的是并行的电路结构，不是顺序的指令；代码有两个读者：仿真器（验证）与综合器（变成门）。
- RTL 抽象回答“每个时钟沿每个寄存器装什么”；`assign` 与 `@(*) always` 块写组合逻辑，`posedge always` 块写时序逻辑。
- `reg` 不是寄存器；组合块分支不赋值会推断出锁存器；敏感列表用 `@(*)`。
- 军规：时序用 `<=`（对应“同时采样旧值”的物理现实），组合用 `=`，绝不混用。
- 状态机用三段式：状态寄存器、次态 `case`、输出逻辑，与状态图一一对应；化简交给综合器。
- 测试平台用不可综合语句制造激励与检查；可综合子集写芯片，全语言写验证。

思考题（答案见附录 A）

- 8.1 若把三段式状态机中“一段”里的 `state <= next` 误写成阻塞赋值 `state = next`，单看这一个块，综合出的电路会变吗？再考虑一段与二段合并、在同一个 `posedge` 块里既算次态又更新状态的写法，为什么这时阻塞与非阻塞的选择就生死攸关了？
- 8.2 下面的组合 `always` 块会推断出什么电路？`always @(*) begin if (en) y = d; end`。请指出问题，并给出两种修复方法。
- 8.3 解释为什么 `#5` 和 `initial` 不可综合，而 `always @(posedge clk)` 可综合。“芯片里没有‘等 5 纳秒’这个门”这句话，严格说漏了什么？（提示：想想第 6 章，芯片里“时间”由谁定义。）
- 8.4 写一个模 10 计数器（0 数到 9 后回 0）的 Verilog 模块，带低电平有效异步复位。想一想：如果忘了处理“数到 9”这个边界，电路会数到多少才回绕？为什么？

第九章

从代码到硅片：设计流程全景

前八章搭起了从电压到状态机、从门到语言的完整阶梯。最后一章登高望远：一段 Verilog 代码，要经过怎样的旅程，才变成指甲盖上集成上百亿晶体管的芯片？本章不再深挖某一项技术，而是把旅程的每一站串起来，让你看清前面所学的每样东西在真实工业流程中的位置，以及一个此前只是暗示过的事实：这条旅程中，最贵的不是设计，而是“确认设计没错”。

9.1 鸟瞰：一块芯片的旅程

图 9.1 是数字芯片设计的主干流程。它就像一条单向的河：越往下游，修改的代价越高，到了“流片”这一站，回头的成本以千万元和季度计。

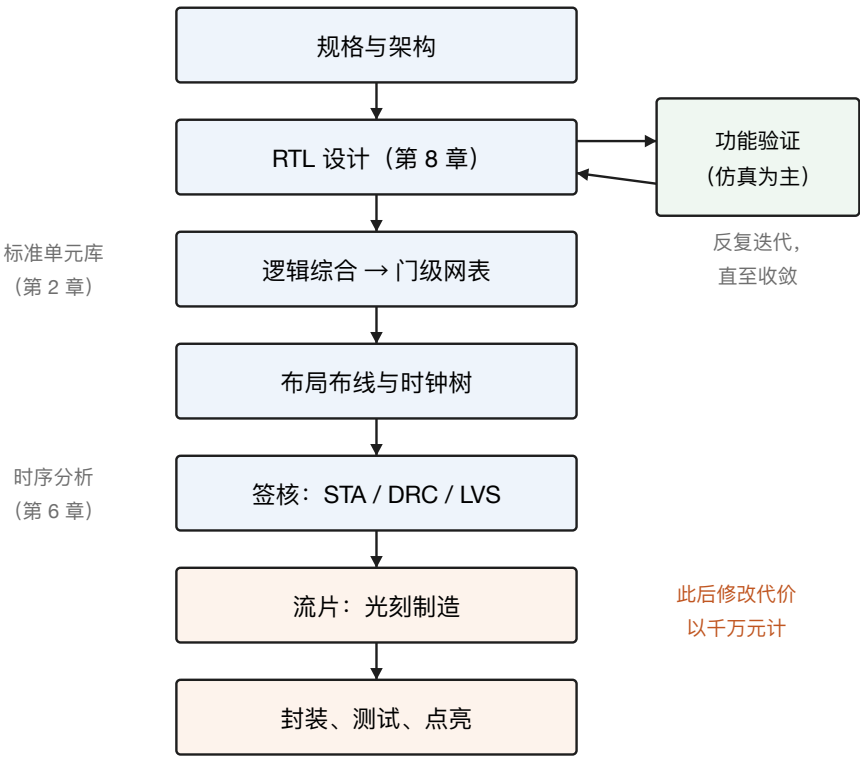


图 9.1 数字芯片设计主干流程。左侧竖流是“把设计变成硅片”，右侧的验证环与设计并行奔跑。下面逐站走一遍。

规格与架构。回答“做什么、做到多少”：功能列表、性能指标、功耗预算、成本上限。架构师把大目标切成模块，定好各模块的接口与指标。这一步的失误最昂贵，因为它的错误会被后面每一步忠实地放大。

RTL 设计。工程师用 Verilog 等语言写出每个模块，即第 8 章的工作。产出是一大批“寄存器加组合云”的描述。

逻辑综合。综合器读入 RTL 与目标工艺的**标准单元库**（第 2 章提过：厂商预先设计好的几百种门、触发器单元，各有精确的延迟、面积、功耗档案），把代码翻译、优化成一张**门级网表**：一份“用了哪些单元、谁连着谁”的清单。第 3 章的化简、第 4 章的面积换时间，综合器都替你做，而且带着时序约束做：你告诉它目标频率，它对关键路径重点优化。

布局布线。网表还只是拓扑，现在要落到几何：几亿个单元在硅片上各安其位（布局），再用十几层金属导线把它们连起来（布线）。同时构建**时钟树**：让时钟以尽量小的偏斜到达几十万个触发器（第 6 章的伏笔在此收束）。从这一步起，每根导线的长度都变成了真实的延迟。

签核。流片前的最后关卡，几件大事：**静态时序分析**（STA）检查全芯片每一条路径的建立、保持约束，就是第 6 章那两个不等式，乘以几千万条路径；**设计规则检查**（DRC）确认几何图形符合制造厂的工艺规则（线宽、间距等）；**版图与原理图一致性检查**（LVS）确认画出来的版图与网表在电学上是同一个电路。全部通过，生成最终版图文件（GDSII），移交制造。

流片与封测。制造厂用光刻技术把版图一层层“印”到硅圆片上：紫外光透过掩膜版把图形转移到光刻胶，再经蚀刻、掺杂、沉积，循环数百步、历时两三个月。制造的物理与化学是另一整个学科，本书按下不表。圆片切割成裸片，封装进外壳，再由自动测试机逐颗筛选（制造总有瑕疵，合格比例称为**良率**）。最后，第一批芯片回到设计团队手中上电“点亮”，那是芯片工程师职业生涯中最紧张的时刻。

9.2 验证：为什么它是大头

流程图右侧那个不起眼的“功能验证”环，在真实项目中吞掉**六到七成**的人力与时间，常常比设计本身的队伍更庞大。原因藏在一道简单的数学题里。

数学加油站 组合爆炸：为什么不能“全试一遍”

n 个输入位就有 2^n 种输入组合，这种“每加一位、工作量翻倍”的增长称为**指数增长**，它失控的速度远超直觉。一个 64 位加法器，输入共 129 位（两个操作数加进位），组合数 $2^{129} \approx 6.8 \times 10^{38}$ 。假设每秒能试 10^{12} 种（这已是极快的仿真），试完需要

$$\frac{6.8 \times 10^{38}}{10^{12} \times 3.15 \times 10^7 \text{ 秒/年}} \approx 2 \times 10^{19} \text{ 年}$$

约为宇宙年龄的十亿倍。而这只是一个加法器；带状态的电路还要乘上状态空间。结论：**穷举验证在物理上不可能**，验证注定是一门“花有限的力气，博最大的信心”的学问。

于是验证工程师的武器库都围绕“聪明地抽样”与“数学地证明”展开：**定向测试**瞄准边界情形（进位链全长传播、计数器回绕）；**随机激励**大面积撒网，配合**覆盖率统计**告诉你哪些角落还没探过；**断言**把设计意图写成可自动检查的命题（“请求发出后八拍内必有应答”），仿真中时刻监督；**形式验证**则动用数学推理，在不跑任何激励的情况下证明某性质对所有输入成立。即便如此，验证也只能逼近而永远无法宣称完美，何时“足够放心”是一个工程与经济的判断。

为什么如此舍得砸人力？因为流片是单向门：先进工艺一套掩膜版数以千万元计，制造周期数月；芯片回来才发现功能错误，多数只能改版重来（respin），产品窗口一并错过。在仿真里多找到一个反例，可能就省下一次千万元级的学费，这笔账驱动着整个行业的验证文化。

9.3 PPA：工程的三角权衡

贯穿整条流程，所有决策最终都投射到三个指标上：**性能**（Performance，频率或吞吐率）、**功耗**（Power）、**面积**（Area，直接决定成本），合称**PPA**，如图 9.2。

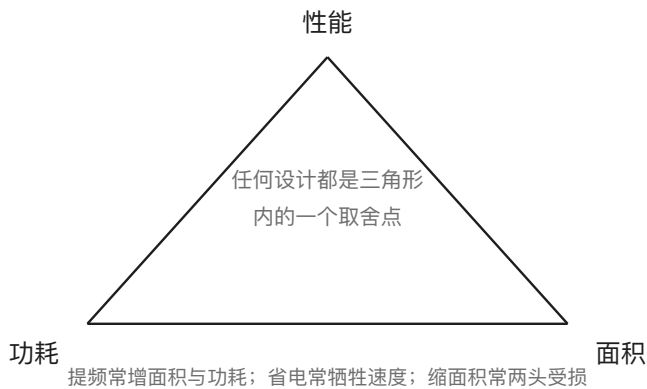


图 9.2 PPA 三角。本书处处是它的影子：超前进位与流水线用面积、功耗换性能；降压省电则牺牲速度。

回望全书，几乎每个技术抉择都是这个三角形上的移动：超前进位用面积买时间（第 4 章），流水线用寄存器与功耗买吞吐（第 6 章），独热编码用触发器买速度（第 7 章），降低电压用速度买功耗（第 2 章）。没有“最好的设计”，只有“最适合这份规格的取舍”。工程与科学的分野，正在于此。

9.4 结语：你已经走到哪里

请允许本书用一分钟收拢九章的线索。第 1 章，我们把连续的电压切成 0 与 1，用“放弃精度”换来绝对可靠；第 2 章，晶体管开关组成互补的门，抽象的高塔开始奠基；

第3章，布尔代数让电路可以纸上推演；第4章，选择与加法两块积木，透露了“逻辑即查表”与“速度即路径”两个天机；第5章，反馈环诞生记忆，时钟把时间切成节拍；第6章，两个不等式统治了整个同步世界；第7章，状态机让电路按剧本行事；第8章，一切装进语言，交给工具；第9章，代码顺流而下，凝固成硅。

从沙子到思维机器，中间隔着的不是魔法，而是一层层朴素得可以在本小书里讲完的抽象。这是数字设计最动人的地方：每一层都简单，叠起来便通天。

往深处走的路标如下，都经过时间检验：

- 体系结构：Harris & Harris, 《数字设计和计算机体系结构》：与本书衔接最顺，后半部带你亲手搭一个能跑程序的处理器；随后可读 Patterson & Hennessy 的《计算机组成与设计》。
- 电路与物理实现：Weste & Harris, 《CMOS 超大规模集成电路设计》：把第2章与第9章之间的所有细节补齐。
- 语言与实践：在 HDLBits 网站上把 Verilog 练到指尖；用开源工具 Icarus Verilog 或 Verilator 仿真，用 Yosys 体验综合。
- 集大成的练习：读懂一个开源 RISC-V 小核（如 picorv32），然后，写一个你自己的。到那一天，这本小书的使命便完成了。

本章小结

- 主干流程：规格 → RTL → 综合（标准单元库）→ 布局布线与时钟树 → 签核（STA、DRC、LVS）→ 流片 → 封测点亮；越往下游改错越贵，流片是单向门。
- 组合爆炸使穷举验证在物理上不可能；定向、随机加覆盖率、断言、形式验证是“抽样与证明”的组合拳；验证吞掉项目大半人力，因为仿真里的反例远比改版便宜。
- STA 不依赖激励，把第6章的两个不等式检查到每一条路径。
- 一切设计决策归于 PPA 三角：性能、功耗、面积不可兼得，规格决定取舍点。

思考题（答案见附录 A）

- 9.1 时序为什么靠静态时序分析签核，而不是“多跑些仿真看看会不会错”？从覆盖率与激励依赖两个角度说明。
- 9.2 用本书学过的实例，各举一个“牺牲面积换性能”“牺牲性能换功耗”“牺牲功耗换性能”的设计决策，并简述其机理。
- 9.3 一个 32 位乘法器共 64 位输入。按每秒仿真 10^9 种组合计算，穷举需要多少年？这个数字告诉我们验证策略必须建立在什么之上？
- 9.4 有一颗芯片在签核时发现某条捷径路径保持时间违例 20 ps，项目经理问：“能不能先流片，回来降频用？”请引用第6章的结论作答，并解释保持违例为什么必须在流片前修复。

附录 A 思考题参考答案

答案给出的是主干思路与关键结果。若你的推导路径不同但结论一致，完全正常；若结论不同，请回到对应章节找出分歧的第一步，那里往往就是收获所在。

第 1 章

1.1 模拟电视里，画质与信噪比连续挂钩：信号每弱一分，雪花就多一分，劣化是**渐变**的。数字电视里，只要噪声不超过噪声容限（加上纠错码的补救余量），每个比特都被完美判读，画面与满信号时毫无区别；一旦噪声越过界限，误码率急剧上升、纠错失效，系统瞬间从“完美”跌到“不可用”。数字抽象把渐变的物理劣化转换成了阶跃式的逻辑失效，这就是“悬崖效应”。

1.2 $NM_L = V_{IL} - V_{OL} = 0.9 - 0.2 = 0.7 \text{ V}$ ； $NM_H = V_{OH} - V_{IH} = 3.1 - 2.4 = 0.7 \text{ V}$ 。想增大容限，可以把输出做得更“极端”（降低 V_{OL} 、抬高 V_{OH} ，代价是更强的驱动、更大的功耗或更慢的速度），或把输入判读放得更宽（抬高 V_{IL} 、降低 V_{IH} ，代价是禁区变窄，判决电路更难做、更易误判），或整体提高电源电压（代价是动态功耗按 V^2 上涨，见第 2 章）。

1.3 -1 的补码是 $256 - 1 = 255 = (11111111)_2$ ，即全 1。道理：全 1 加 1 后溢出回绕到 0，所以这个码字满足 $x + 1 \equiv 0$ ，它就是 -1 。范围不对称是因为 256 个码字里，最高位为 1 的 128 个分给了负数（ -1 到 -128 ），最高位为 0 的 128 个要容纳 0 与正数，0 占掉一格，正数只剩 127 格。 $+128$ 与 -128 对应同一个码字 10000000，约定判给负方。

1.4 二进制：两个区间加一个禁区，粗略地 $3w = 3.3 \text{ V}$ ，每段约 1.1 V。三进制：三个区间加两个禁区， $5w = 3.3 \text{ V}$ ，每段约 0.66 V，抗噪能力降到六成。收益是每位携带 1.58 比特，位数省约 37%。再算上“开关”天然只有开、关两态，产生和判读中间电平需要额外电路，这笔交易在工程上并不划算，这正是三进制计算机没有流传下来的原因。

第 2 章

2.1 下拉网络：三个 NMOS 串联（ A 、 B 、 C 全为 1 才拉低）；上拉网络：三个 PMOS 并联（任一为 0 即拉高）。两个网络导通条件互补，共 6 个晶体管。

2.2 输入停在 $V_{DD}/2$ 附近时，NMOS 与 PMOS **同时半导通**：其一，从电源到地出现一条持续的直通电流路径，静态功耗剧增、器件发热；其二，输出电压停在中间禁区，下游各门可能判读不一，行为不可预测。悬空输入还会因感应电荷缓慢漂移，随机地在两态间游走。因此 CMOS 的输入必须时刻接在确定电平上，这也是“稳定信号不得停留禁区”的电路层理由。

2.3 四个与非门的经典构造：设 $N = \overline{AB}$ （第一个门），则

$$Y = \overline{A \cdot N \cdot B \cdot N}$$

即第二、三个门分别算 $\overline{AN}$ 与 $\overline{BN}$ ，第四个门把两者与非。展开可验证 $Y = A\overline{B} + \overline{A}B = A \oplus B$ 。

2.4 动态功耗比例为 $(0.7/1.0)^2 \times (2/3) \approx 0.49 \times 0.667 \approx 0.33$ ，约为原来的三分之一。这正是移动芯片的**动态电压频率调节**（DVFS）策略：负载轻时同时降频降压，功耗大幅下降；其中降压贡献了平方级的大头。

第 3 章

3.1 真值表 8 行逐一核对，两边处处相等（略）。代数法从右往左：

$$\begin{aligned}(A + B)(A + C) &= AA + AC + AB + BC = A + AC + AB + BC \\ &= A(1 + C + B) + BC = A + BC\end{aligned}$$

依次用了分配律、幂等律（ $AA = A$ ）与支配律（ $1 + \dots = 1$ ）。

3.2 两次德摩根：

$$\overline{AB + C} = \overline{AB} \cdot \overline{C} = (\overline{A} + \overline{B}) \overline{C}$$

3.3 卡诺图（行为 AB ，列为 CD ，均按格雷码序 00、01、11、10）：

$AB \setminus CD$	00	01	11	10
00	1	0	0	1
01	0	1	1	0
11	0	1	1	0
10	1	0	0	1

中央四格（ $B = 1, D = 1$ ）圈成一个四格块，得 BD ；四个角靠上下、左右环绕互为相邻，也是一个四格块（ $B = 0, D = 0$ ），得 $\overline{B}\overline{D}$ 。故

$$F = BD + \overline{B}\overline{D}$$

即“ B 与 D 相同则为 1”，正是异或的反（同或）。

3.4 用归纳法。 $n = 2$ 时 $A_1 \oplus A_2$ 恰在“有奇数个 1”（即恰一个 1）时为 1，成立。设 $n = k$ 时结论成立，则 $Y_{k+1} = Y_k \oplus A_{k+1}$ ：若 $A_{k+1} = 0$ ，奇偶性与判断都不变；若 $A_{k+1} = 1$ ，1 的个数奇偶翻转，输出也恰好翻转。归纳成立。这个电路叫**奇偶校验器**（parity checker）：发送或存储数据时附加一个校验位，使全体 1 的个数为偶（或奇）；接收端重算校验，即可发现任何**单比特**错误。内存的校验位、串口的奇偶校验位都是它。

第 4 章

4.1 非门：选择端接 A ，数据端 $D_0 = 1$ 、 $D_1 = 0$ ，则 $Y = \overline{A}$ 。与门：选择端接 A ， $D_0 = 0$ 、 $D_1 = B$ ，则 $Y = AB$ 。或门：选择端接 A ， $D_0 = B$ 、 $D_1 = 1$ ，则 $Y = A + B$ 。既能造出功能完备的{非、与、或}，二选一 mux（加常数）便是功能完备的。

4.2 左边展开：

$$AB + C_{in}(A\overline{B} + \overline{A}B) = AB + A\overline{B}C_{in} + \overline{A}BC_{in}$$

利用 AB 项可以免费吸纳 ABC_{in} （因 $AB + ABC_{in} = AB$ ），把它补进去两次：

$$\begin{aligned} &= AB + (A\overline{B}C_{in} + ABC_{in}) + (\overline{A}BC_{in} + ABC_{in}) = AB + AC_{in}(\overline{B} + B) + BC_{in}(\overline{A} + A) \\ &= AB + AC_{in} + BC_{in} \end{aligned}$$

4.3 行波： $64 \times 2 \times 25 = 3200 \text{ ps} = 3.2 \text{ ns}$ ，最高约 $1/3.2 \text{ ns} \approx 312 \text{ MHz}$ （忽略触发器开销）。理想超前进位： $6 \times 25 = 150 \text{ ps}$ ，约 6.7 GHz ，速度提高约 21 倍。实际的分组超前进位达不到这么理想，但数量级的差距足以说明为什么高速加法器无一例外抛弃纯行波结构。

4.4 sub = 1 时：每个异或门计算 $B_i \oplus 1 = \overline{B_i}$ ，即把 B 按位取反； $C_0 = 1$ 恰好再加 1。于是电路计算 $A + \overline{B} + 1$ ，按第 1 章补码恒等式，这正是 $A - B$ （模 2^n ）。sub = 0 时： $B_i \oplus 0 = B_i$ ， $C_0 = 0$ ，电路就是普通加法 $A + B$ 。一个信号同时完成“取反”与“加一”两项差事，是补码优雅性的集中体现。

第 5 章

5.1 假设存在稳定状态，设某节点为 Q ：绕环一周经过三次取反，回到本节点应等于 $\overline{Q}$ ，与假设矛盾，故不存在稳定状态。电路会永不停歇地振荡：每个节点以“三级门延迟”为半周期翻转，这就是**环形振荡器**。它是有用的电路：可充当片上时钟源，更常用来测量工艺速度（数它的振荡频率就知道这批晶体管有多快）。

5.2 $S = R = 1$ 时两个或非门输出都为 0，反馈环处于“两边都是 0”的对称状态。两输入恰好同时撤回 0 的一瞬，两个门都看到输入全 0、都要把输出抬高；而任何一边先抬起来又会把对方压回去。电路落在一个完全对称的不稳定平衡点上，最终倒向哪边取决于两个门的微小速度差和噪声，可能短暂振荡或悬在中间电平。这正是“山顶的球”：禁用组合是一个人为制造亚稳态的按钮，所以必须禁用。

5.3 使能为高的窗口内，链上所有锁存器同时透明，整条链变成一根组合导线：数据能一口气穿过几级，取决于窗口宽度与每级传播延迟之比，完全失控。把窗口改窄不能根治：只要窗口宽于一级的传播延迟，就可能穿透两级；窄到接近传播延迟，又会侵犯采样所需的稳定时间（类似建立时间），数据反而存不进去。“一段时间”与“一个瞬间”之间有本质鸿沟，必须靠主从结构用两个互不重叠的窗口把它填平。

5.4 一级同步器的输出可能仍处亚稳，直接被内部逻辑消费，留给它落定的时间只有本周期内组合路径的富余量，MTBF 不够看。第二级让第一级享有**接近一个整周**

期的落定时间，MTBF 乘上 $e^{T/\tau}$ 这个天文数字。再加第三级，MTBF 再乘一次同样的因子，代价是输入到可用多等一拍、多一个触发器；多数场合两级已绰绰有余。时钟频率翻倍则 T 减半，指数上的 T/τ 减半，MTBF 变成原来的平方根（急剧恶化），所以越高频的设计越要认真对待同步器，必要时用三级。

第 6 章

6.1 建立约束： $T \geq 60 + 800 + 50 = 910 \text{ ps}$, $f_{\max} \approx 1.10 \text{ GHz}$ 。保持约束： $t_{\text{cq}} + t_{\text{logic}}^{\min} = 60 + 20 = 80 \geq t_h = 30$ ，满足，无需修复。若最短路径再快些（如 $t_{\text{logic}}^{\min}$ 趋近 0 而 t_h 更大）导致违例，标准修法是在该短路径上串入缓冲器增加延迟。

6.2 建立违例：最慢路径在当前周期内赶不到，把周期拉长（降频）即可消除，芯片仍能工作只是变慢，故是性能问题。保持违例：同一个时钟沿前后的赛跑，约束里没有周期项，任何频率下都错，芯片直接报废，故是功能问题。修复：建立违例靠优化慢路径（化简逻辑、加流水线、换快单元）或降频；保持违例只能在流片前给快路径添延迟（插缓冲器）。

6.3 $T(n) = 1200/n + 80 \text{ ps}$, $f(n) = 1/T(n)$: $n = 1$: 1280 ps , 0.78 GHz ; $n = 2$: 680 ps , 1.47 GHz ; $n = 4$: 380 ps , 2.63 GHz ; $n = 8$: 230 ps , 4.35 GHz 。频率提升倍率依次约 1.9、1.8、1.7 倍，逐次递减，因为固定开销 80 ps 的占比从 6% 涨到 35%。现实中的额外障碍：逻辑很难切得绝对均匀（最慢一级封顶）；流水级之间存在数据相关，深流水需要停顿或旁路机制，实际吞吐率打折扣；寄存器与时钟树的功耗、面积水涨船高。

6.4 FF2 的时钟晚到 δ ，相当于给数据多 δ 的赶路时间、也让“旧数据只需保住到沿后 t_h ”这条线整体后移。两式变为：建立： $T_{\text{clk}} + \delta \geq t_{\text{cq}} + t_{\text{logic}}^{\max} + t_{\text{su}}$ （放松了）；保持： $t_{\text{cq}} + t_{\text{logic}}^{\min} \geq t_h + \delta$ （收紧了）。 δ 以相同符号出现在两条约束的相反侧：接收端时钟每晚到一分，建立就宽一分、保持就紧一分；偏早则相反。所以偏斜永远是双刃剑，时钟树的目标是把它压到最小而不是利用它。

第 7 章

7.1 按编码展开状态表（ $Q_1 Q_0$ 为现态， X 为输入）：

Q_1	Q_0	X	D_1	D_0	Z
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	1	0	0
0	1	1	0	1	0
1	0	0	0	0	0
1	0	1	1	1	0
1	1	0	1	0	1
1	1	1	0	1	1

D_0 列与 X 列完全相同, 故 $D_0 = X$ 。 D_1 的 1 出现在 (Q_1, Q_0, X) 为 $(0, 1, 0)$ 、 $(1, 0, 1)$ 、 $(1, 1, 0)$ 三处: $(0, 1, 0)$ 与 $(1, 1, 0)$ 相邻合并为 $Q_0\overline{X}$, $(1, 0, 1)$ 单独成项 $Q_1\overline{Q_0}X$, 故 $D_1 = Q_0\overline{X} + Q_1\overline{Q_0}X$ 。 Z 仅在 $Q_1 = Q_0 = 1$ 时为 1, $Z = Q_1Q_0$ 。与正文一致。

7.2 不可重叠意味着报告之后已消耗的位不得复用, 匹配从零重新开始。只需改 S_3 的出边: $X = 0$ 时去 S_0 (原来去 S_2 ; 因为新历史只有一个“0”, 不是任何前缀); $X = 1$ 时仍去 S_1 (新历史“1”本身就是新前缀, 并未复用旧位)。修改后输入“10101”只在第 3 拍报告一次: 第 4 拍从 S_3 回 S_0 , 第 5 拍走到 S_1 , 不再触发。状态表除 S_3 行的 $X = 0$ 列改为 S_0 外, 其余不变。

7.3 状态: S_0 一无所获; S_1 已匹配“1”; S_2 已匹配“11”; S_3 刚凑齐“110”, 输出 1。转移: $S_0: 0 \rightarrow S_0, 1 \rightarrow S_1$; $S_1: 0 \rightarrow S_0$ (“10”不是“110”的前缀), $1 \rightarrow S_2$; $S_2: 0 \rightarrow S_3$ (凑齐), $1 \rightarrow S_2$ (“111”的末两位“11”仍是前缀); $S_3: 0 \rightarrow S_0$ (末位“00”无用), $1 \rightarrow S_1$ (新“1”是前缀)。同样是 4 个状态: 状态数由“前缀进度”决定 (模式长 3, 进度 0 到 3), 与具体模式无关。

7.4 二进制编码需要 $\lceil \log_2 12 \rceil = 4$ 个触发器; 独热需要 12 个。独热编码下“处于 S_5 ”就是第 5 号触发器的输出线本身, 条件即一个二输入与门: $Q_5 \cdot r$ 。二进制编码则要先做一次 4 位译码 ($Q_3\overline{Q_2}Q_1\overline{Q_0}$ 之类的四输入与)。这说明独热的优势: 状态判断零解码、输出逻辑浅而快, 代价是触发器多, 在寄存器便宜的 FPGA 上尤其合算。

第 8 章

8.1 单看一段块, 块内只有一条赋值语句, 不存在“前面的结果被后面使用”的问题, 阻塞与非阻塞综合出同样的触发器; 但仿真时若另一个 posedge 块恰好读 state, 阻塞赋值会使读到新值还是旧值取决于仿真器调度顺序 (竞态)。若把次态计算与状态更新合并进同一个块并使用阻塞赋值, 本拍算出的 next 会立即写穿 state, 后续语句 (以及依赖 state 的判断) 读到的已是“未来”, 行为等价于数据在一拍内多走一步, 与“所有触发器同时采样旧值”的硬件现实脱节, 可能综合出与仿真不一致的电路。军规“时序用 $\leq$ ”正是为了封死这一整类事故。

8.2 $en = 0$ 时 y 没有被赋值，语义为“保持旧值”，综合器将推断出一个电平敏感的 D 锁存器 (en 当使能)。若本意是纯组合逻辑，修复办法：补全分支 ($else\ y = 1'b0;$)，或在块首写默认值 ($y = 1'b0;$ $if\ (en)\ y = d;$)。若本意确实要存储，则应改用 $posedge$ 时钟的触发器，而不是靠疏漏得到的锁存器。

8.3 $\#5$ 与 $initial$ 是对仿真器时间轴的指令：仿真器有一个全局时钟表可以“等 5 纳秒”“只执行一次”，而硅片上只有导线与门，没有执行语句的机器。 $always\ @(posedge\ clk)$ 可综合，因为它描述的不是动作序列，而是一种结构对事件的响应，恰好与 D 触发器这个器件对应。严格说，芯片里并非做不出延时：可以用计数器数 N 个时钟拍（在时钟定义的离散时间里等待），这正是第 6 章的要点：芯片内的时间以时钟节拍为单位构造，缺的不是延时能力，而是“以绝对秒为单位”的原语。

8.4 参考实现：

```
module mod10 (
    input wire      clk,
    input wire      rst_n,
    output reg [3:0] cnt
);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            cnt <= 4'd0;
        else if (cnt == 4'd9) cnt <= 4'd0;    // 边界：数到 9 回 0
        else
            cnt <= cnt + 4'd1;
    end
endmodule
```

若忘记 $cnt == 9$ 的边界处理，4 位寄存器会按自然的模 16 回绕：0 数到 15 才回 0。原因见第 1 章： n 位加法天生是模 2^n 算术，模 10 是“人为”的回绕，必须显式写出。

第 9 章

9.1 仿真的结论只覆盖“跑过的激励”，而路径是否被激活、慢路径是否被踩中，取决于具体输入序列；组合爆炸（9.2 节）决定了激励永远只是沧海一粟，用它验证时序等于抽查。STA 则完全不依赖激励：对网表上每一条时序路径，用单元库给出的延迟上下界直接检验第 6 章的两个不等式，一次性覆盖百分之百的路径。代价是分析偏保守（可能报出实际不会同时发生的“伪路径”，需人工排除），但“保守而完备”正是签核想要的性质。

9.2 牺牲面积换性能：行波加法器换成超前进位（第 4 章），门数增加数倍，关键路径从 $O(n)$ 缩到常数级。牺牲性能换功耗：DVFS 降压降频（第 2 章思考题 2.4），速度下降换来平方级的功耗节省。牺牲功耗换性能：流水线（第 6 章），插入成排寄存

器及其时钟翻转功耗，换取数倍的吞吐率；或者给关键路径换用更大尺寸、更快但更耗电的单元。

9.3 $2^{64} \approx 1.8 \times 10^{19}$ 种组合，每秒 10^9 种需要 1.8×10^{10} 秒 ≈ 585 年。这还只是 32 位乘法器一个纯组合模块。结论：验证策略必须建立在**抽样与证明**之上，即用定向测试打边界、随机测试加覆盖率撒网、断言实时监督、形式验证做数学证明，而不能指望穷举。

9.4 答：不能。降频修改的是时钟周期 T_{clk} ，而保持约束 $t_{\text{cq}} + t_{\text{logic}}^{\min} \geq t_h$ 中根本不含 T_{clk} （第 6 章）：它是同一个时钟沿前后新旧数据的赛跑，频率再低，这场赛跑照样输。保持违例的芯片在任何频率下都会把旧数据冲掉，属于功能性缺陷，流片即报废。必须在签核阶段修复：在该捷径路径上插入缓冲器，把 $t_{\text{logic}}^{\min}$ 垫高 20 ps 以上再流片。

全书完。愿你从此看任何一块芯片，都能看见它体内
那一层层朴素而精确的抽象，以及把它们叠起来的人。